

Distributed Key-Value Store

Complete Experimental Results

with Exhaustive Analysis

Exercise 3 – Final Report

Course: Distributed Computing Fundamentals

Instructor: Dr. Mohammadreza Shournia

Head TA: Pouya Jamshidi

Assistants: Mobina Haghzadeh

M.M. Ebrahim Soltan

Students: Taha Majlesi – 810101504

Alireza Karimi – 810101492

Semester: Spring 2025 (1405)

University: University of Tehran

Department: Electrical and Computer Engineering

July 24, 2026

Contents

1	Abstract	7
2	Acknowledgments	8
3	Reading Guide and Report Navigation	9
3.1	How to Read This Report	9
3.2	Notation Conventions	9
3.3	Data Availability	9
4	Test Configurations and Methodology	10
4.1	Complete Configuration Matrix	10
4.2	Test Scenarios Overview	10
5	Historical Context and Evolution of Consistency Models	11
5.1	The Pre-Internet Era: Single-Node Databases	11
5.2	The Internet Era: The Rise of Distribution	11
5.3	Key Milestones in Consistency Research	11
5.4	From Theory to Practice	12
6	Scenario 1: Temporary Inconsistency Observation – Complete Results	13
6.1	Overview of Scenario 1	13
6.2	Configuration 1: Eventual Consistency, 0ms Delay	13
6.2.1	Raw Test Data	13
6.2.2	Exhaustive Analysis	13
6.2.3	Key Takeaway for This Configuration	14
6.3	Configuration 2: Eventual Consistency, 500ms Delay	14
6.3.1	Raw Test Data	14
6.3.2	Exhaustive Analysis	15
6.3.3	Key Takeaway for This Configuration	16
6.4	Configuration 3: Eventual Consistency, 2000ms Delay	16
6.4.1	Raw Test Data	16
6.4.2	Exhaustive Analysis	16
6.4.3	Key Takeaway for This Configuration	17
6.5	Configuration 4: Strong Consistency, 0ms Delay	17
6.5.1	Raw Test Data	17
6.5.2	Exhaustive Analysis	18
6.5.3	Key Takeaway for This Configuration	18
6.6	Configuration 5: Strong Consistency, 500ms Delay	19
6.6.1	Raw Test Data	19
6.6.2	Exhaustive Analysis	19
6.6.3	Key Takeaway for This Configuration	20
6.7	Configuration 6: Strong Consistency, 2000ms Delay	20
6.7.1	Raw Test Data	20
6.7.2	Exhaustive Analysis	20
6.7.3	Key Takeaway for This Configuration	21
6.8	Scenario 1: Comprehensive Summary Table	21

7	Detailed Protocol Specifications	22
7.1	The PUT Protocol: Step-by-Step Wire Format	22
7.1.1	Client to Replica: PUT Request	22
7.1.2	Replica to Client: PUT Response (Success)	22
7.1.3	Replica to Client: PUT Response (Failure – Strong Consistency)	22
7.1.4	Replica to Replica: Replication Request	23
7.1.5	Replica to Replica: Replication Response	23
7.2	The GET Protocol: Step-by-Step Wire Format	23
7.2.1	Client to Replica: GET Request	23
7.2.2	Replica to Client: GET Response (Found)	24
7.2.3	Replica to Client: GET Response (Not Found)	24
7.3	The Health Check Protocol	24
7.4	The Stop/Start Protocol (Failure Simulation)	24
8	Scenario 2: Replica Failure Behavior – Complete Results	25
8.1	Overview of Scenario 2	25
8.2	Eventual Consistency (0ms Delay) – Failure Test	25
8.2.1	Raw Test Data	25
8.2.2	Exhaustive Analysis	25
8.2.3	Key Takeaway	26
8.3	Strong Consistency (0ms Delay) – Failure Test	26
8.3.1	Raw Test Data	26
8.3.2	Exhaustive Analysis	27
8.3.3	Key Takeaway	27
8.4	Scenario 2: Summary	27
9	Concurrency Model and Thread Safety Analysis	28
9.1	Go’s Concurrency Primitives Used	28
9.2	Thread Safety Proof for the Data Store	28
9.3	Potential Concurrency Issues and Mitigations	28
9.3.1	Issue 1: Deadlock Prevention	28
9.3.2	Issue 2: Lock Contention Under High Load	29
9.3.3	Issue 3: Goroutine Leak	29
10	Scenario 3: Concurrent Conflict Resolution – Complete Results	30
10.1	Overview of Scenario 3	30
10.2	Eventual Consistency (0ms Delay) – Conflict Test	30
10.2.1	Raw Test Data	30
10.2.2	Exhaustive Analysis	30
10.2.3	Key Takeaway	31
10.3	Strong Consistency (0ms Delay) – Conflict Test	31
10.3.1	Raw Test Data	31
10.3.2	Exhaustive Analysis	32
10.3.3	Key Takeaway	32
10.4	Scenario 3: Summary	32

11 Scenario 4: Network Delay Impact – Complete Results	33
11.1 Overview	33
11.2 Eventual Consistency, 0ms Delay	33
11.2.1 Raw Data	33
11.2.2 Analysis	33
11.3 Eventual Consistency, 500ms Delay	33
11.3.1 Raw Data	33
11.3.2 Analysis	33
11.4 Eventual Consistency, 2000ms Delay	33
11.4.1 Raw Data	33
11.4.2 Exhaustive Analysis	34
11.5 Strong Consistency, 0ms Delay	34
11.5.1 Raw Data	34
11.5.2 Analysis	35
11.6 Strong Consistency, 500ms Delay	35
11.6.1 Raw Data	35
11.6.2 Analysis	35
11.7 Strong Consistency, 2000ms Delay	35
11.7.1 Raw Data	35
11.7.2 Analysis	35
11.8 Scenario 4: Master Summary Table	35
12 Statistical Analysis of Experimental Results	36
12.1 Measurement Confidence and Error Analysis	36
12.1.1 Sources of Measurement Error	36
12.1.2 Confidence in Key Measurements	36
12.2 Reproducibility Analysis	36
12.3 Statistical Summary of All Measurements	37
13 Overall Analysis and Conclusions	38
13.1 The CAP Theorem Empirically Validated	38
13.2 Quantitative Trade-off Formula	38
13.3 Recommendations	38
13.4 Limitations and Future Work	38
14 Visual Analysis of Experimental Results	39
14.1 Chart 1: PUT Latency Comparison Across All Configurations	39
14.1.1 Chart Structure Explanation	39
14.1.2 Detailed Visual Analysis	39
14.1.3 Key Insight from This Chart	41
14.2 Chart 2: Stale Reads Comparison	42
14.2.1 Chart Structure Explanation	42
14.2.2 Detailed Visual Analysis	42
14.2.3 Key Insight from This Chart	43
14.3 Chart 3: Convergence Time Comparison	44
14.3.1 Chart Structure Explanation	44
14.3.2 Detailed Visual Analysis	44
14.3.3 Key Insight from This Chart	45
14.4 Chart 4: PUT Latency vs Network Delay (Line Chart)	46

14.4.1	Chart Structure Explanation	46
14.4.2	Detailed Visual Analysis	46
14.4.3	Key Insight from This Chart	47
14.5	Chart 5: Stale Reads vs Network Delay (Line Chart)	48
14.5.1	Chart Structure Explanation	48
14.5.2	Detailed Visual Analysis	48
14.5.3	Key Insight from This Chart	49
14.6	Chart 6: Radar Chart – Multi-Dimensional Comparison	50
14.6.1	Chart Structure Explanation	50
14.6.2	Detailed Visual Analysis	51
14.6.3	Key Insight from This Chart	52
14.7	Chart 7: Dual Radar Charts – Side-by-Side Comparison	53
14.7.1	Chart Structure Explanation	53
14.7.2	Detailed Visual Analysis	53
14.7.3	Key Insight from This Chart	54
14.8	Chart 8: Grouped Bar Chart – All Metrics Together	55
14.8.1	Chart Structure Explanation	55
14.8.2	Detailed Visual Analysis	55
14.8.3	Key Insight from This Chart	56
14.9	Chart 9: Convergence Timeline	57
14.9.1	Chart Structure Explanation	57
14.9.2	Detailed Visual Analysis	58
14.9.3	Key Insight from This Chart	59
14.10	Chart 10: Comprehensive Summary Dashboard	60
14.10.1	Chart Structure Explanation	60
14.10.2	Detailed Visual Analysis	61
14.10.3	The Dashboard as a Complete Picture	61
14.10.4	Key Insight from This Chart	62
15	Ethical and Societal Implications of Consistency Choices	63
15.1	When Stale Data Causes Harm	63
15.2	The Environmental Cost of Strong Consistency	63
15.3	Accessibility and the Digital Divide	63
15.4	Privacy Implications	64
16	Final Synthesis: What All Charts Together Reveal	65
16.1	The Unified Narrative	65
16.2	Personal Reflections on the Learning Process	65
16.2.1	The Gap Between Theory and Implementation	65
16.2.2	Debugging Distributed Systems is Hard	65
16.2.3	The Value of Automated Testing	65
16.2.4	Surprising Discoveries	66
16.3	The Inescapable Conclusion	67
16.4	Practical Guidance from the Charts	67
17	Deep Dive Analysis: Replica Failure Scenario	68
17.1	Timeline Reconstruction of Failure Scenario	68
17.1.1	Eventual Consistency Failure Timeline	68
17.1.2	Strong Consistency Failure Timeline	68

17.1.3 What Would Happen With 2 of 3 Replicas Down?	69
18 Deep Dive Analysis: Versioning and Conflict Resolution	71
18.1 Why Simple Version Numbers Work	71
18.1.1 The Version Number Lifecycle	71
18.1.2 Why Version Numbers Are Essential	71
18.2 The Last-Write-Wins Algorithm in Detail	72
18.2.1 When LWW Is Applied	72
18.2.2 The Timestamp Comparison	72
18.2.3 LWW Pseudocode	73
18.3 Experimental Validation of LWW	73
18.3.1 Limitations Revealed by Experiment	73
19 Quantitative Analysis of Network Delay Impact	75
19.1 Mathematical Modeling of Observed Behavior	75
19.1.1 Eventual Consistency Model	75
19.1.2 Strong Consistency Model	75
19.2 Extrapolation to Real-World Scenarios	76
19.3 The Cost-Benefit Analysis	76
20 System Limitations and Proposed Improvements	77
20.1 Identified Limitations of Our Implementation	77
20.1.1 1. In-Memory Storage (No Persistence)	77
20.1.2 2. No Catch-Up Mechanism for Failed Replicas	77
20.1.3 3. Simulated Network Delays (Not Realistic)	78
20.1.4 4. Fixed Three-Replica Configuration	78
20.1.5 5. No Quorum Reads	78
21 Comparison with Production Distributed Systems	79
21.1 Amazon DynamoDB	79
21.2 Apache Cassandra	79
21.3 Google Spanner	80
22 Lessons Learned and Best Practices for Distributed Systems	81
22.1 Design Principles Validated by Our Experiments	81
22.1.1 1. Always Version Your Data	81
22.1.2 2. Plan for Failure from Day One	81
22.1.3 3. Choose Consistency Based on Requirements, Not Ideology	81
22.1.4 4. Test Under Realistic Conditions	81
22.1.5 5. Monitor Everything	82
22.2 Anti-Patterns to Avoid	82
22.2.1 1. Assuming the Network is Reliable	82
22.2.2 2. Using LWW for Everything	82
22.2.3 3. Ignoring the CAP Theorem	82
23 Future Research and Extension Directions	83
23.1 Short-Term Extensions (1-2 Weeks)	83
23.2 Medium-Term Extensions (1-2 Months)	83
23.3 Long-Term Research Questions	83

A	Appendix A: Complete Source Code Architecture	84
A.1	Repository Structure	84
A.2	Key Data Structures	84
A.3	API Route Registration	85
B	Appendix B: Complete Test Execution Log	85
B.1	Test Run Metadata	85
C	Appendix C: Glossary of Terms	86
D	Appendix D: Troubleshooting Guide	87
D.1	Common Issues Encountered During Development	87
E	Executive Summary: Choosing the Right Consistency Model	87
E.1	The One-Paragraph Summary	87
E.2	Decision Matrix	87
E.3	Final Recommendation	87

1 Abstract

This report presents the complete experimental results and exhaustive analysis of a distributed replicated key-value store system. Through 18 automated test executions across 6 distinct configurations (two consistency models $\times$ three network delay settings), we quantitatively demonstrate and thoroughly explain every aspect of distributed system behavior including temporary inconsistency, replica failure handling, concurrent conflict resolution, and network delay impact.

The raw experimental data is presented in its entirety, with each measurement contextualized, analyzed, and connected to fundamental distributed systems theory including the CAP theorem, consistency models, versioning, and conflict resolution strategies. Every metric—PUT latency, GET latency, convergence time, stale read counts, and conflict resolution outcomes—is explained in terms of what it reveals about the underlying system behavior and the broader implications for distributed system design.

2 Acknowledgments

This project was completed as part of the Distributed Computing Fundamentals course at the University of Tehran, Faculty of Electrical and Computer Engineering, under the supervision of Dr. Mohammadreza Shournia. The authors gratefully acknowledge the guidance provided by Head Teaching Assistant Pouya Jamshidi and Exercise Assistants Mobina Haghzadeh and Mohammadmehdi Ebrahim Soltan throughout the design, implementation, and testing phases of this exercise.

The experimental infrastructure was built entirely using open-source tools including the Go programming language, Bash scripting for test automation, Python with matplotlib for data visualization, and LaTeX for report generation. We acknowledge the broader distributed systems community whose foundational work—particularly Brewer’s CAP theorem, Lamport’s Paxos algorithm, and the Dynamo and Spanner papers from Amazon and Google—provided the theoretical framework for this investigation.

3 Reading Guide and Report Navigation

3.1 How to Read This Report

This report is structured to accommodate readers with varying levels of familiarity with distributed systems:

- **For readers seeking a quick overview:** Read the Abstract (Section 1), the Executive Summary (final section), and examine the Summary Dashboard (Chart 10). These provide the key findings, main results, and practical recommendations in approximately 10-15 minutes.
- **For readers interested in theoretical foundations:** Focus on Section 2 (Theoretical Foundations of Distributed Systems), which explains replication motivations, consistency models, client-centric guarantees, and the CAP theorem with detailed examples and mathematical reasoning.
- **For readers interested in system design:** Focus on Section 3 (System Architecture and Design) and Section 4 (Implementation Details), which describe the multi-master architecture, API design, data model, and the algorithms for versioning, replication, and conflict resolution.
- **For readers interested in experimental results:** Focus on Sections 5-7 (Testing Methodology, Experimental Results, and Visual Analysis), which present all raw data, quantitative analysis, and 10 professional charts with exhaustive explanations of every visual pattern.
- **For readers interested in practical implications:** Focus on Section 8 (Overall Analysis), Section 9 (System Limitations and Improvements), and Section 10 (Comparison with Production Systems), which translate experimental findings into actionable recommendations.

3.2 Notation Conventions

Throughout this report, we use consistent notation:

- **Teal/Cyan colored text** indicates Eventual Consistency concepts, behaviors, and results
- **Red/Coral colored text** indicates Strong Consistency concepts, behaviors, and results
- **Bold text** highlights key terms, definitions, and important findings
- **Monospace text** represents code, API endpoints, JSON data, and commands
- Mathematical equations use standard notation: L for latency, d for delay, T for time

3.3 Data Availability

All raw experimental data, source code, configuration files, test scripts, and generated charts are available in the project repository. The complete test output for all 18 test runs across 6 configurations and 4 scenarios can be found in the `results/` directory. The automated test suite script (`run_all_tests.sh`) allows full reproduction of all experimental results.

4 Test Configurations and Methodology

4.1 Complete Configuration Matrix

Six distinct system configurations were tested, spanning the two consistency models and three network delay settings:

Table 1: Complete Test Configuration Matrix

Config	Consistency	Delay	CAP Type	Replication
1	Eventual	0ms	AP	Asynchronous
2	Eventual	500ms	AP	Asynchronous
3	Eventual	2000ms	AP	Asynchronous
4	Strong	0ms	CP	Synchronous Majority
5	Strong	500ms	CP	Synchronous Majority
6	Strong	2000ms	CP	Synchronous Majority

Configuration Rationale:

- **0ms delay:** Baseline measurement showing intrinsic system overhead without artificial latency
- **500ms delay:** Simulates moderate cross-continental network latency (e.g., Europe to North America)
- **2000ms delay:** Simulates extreme network conditions (satellite links, congested networks, multiple hops)
- **Eventual consistency:** Tests AP system behavior (prioritizes Availability over Consistency)
- **Strong consistency:** Tests CP system behavior (prioritizes Consistency over Availability)

4.2 Test Scenarios Overview

Four scenarios were designed to evaluate different aspects of distributed system behavior:

1. **Scenario 1 – Temporary Inconsistency:** Writes a value to one replica and immediately reads from another, measuring whether stale data is returned and how long convergence takes.
2. **Scenario 2 – Replica Failure:** Stops one replica, performs a write, then restarts the replica to observe data recovery behavior.
3. **Scenario 3 – Concurrent Conflict:** Performs simultaneous writes to different replicas for the same key, testing conflict resolution.
4. **Scenario 4 – Network Delay Impact:** Measures convergence time and stale read frequency under varying network delays.

5 Historical Context and Evolution of Consistency Models

5.1 The Pre-Internet Era: Single-Node Databases

Before the widespread adoption of distributed systems, databases operated on single machines. Consistency was simple and absolute: there was only one copy of the data, so reads always returned the most recent write. The ACID properties (Atomicity, Consistency, Isolation, Durability) defined the gold standard for database transactions.

However, single-node databases had fundamental limitations:

- **Single point of failure:** If the machine crashed, the entire system was unavailable
- **Limited scalability:** A single machine can only handle finite load (CPU, memory, I/O)
- **Geographic limitations:** Users far from the data center experienced high latency

5.2 The Internet Era: The Rise of Distribution

The explosive growth of the internet in the late 1990s and early 2000s created new challenges:

- **Global user bases:** Users accessed services from every continent
- **Massive scale:** Services like Google, Amazon, and Facebook served millions of concurrent users
- **Continuous operation:** Users expected 24/7 availability with no downtime
- **Data explosion:** Datasets grew beyond what a single machine could store

These challenges drove the adoption of distributed architectures: spreading data and computation across multiple machines. However, distribution introduced the consistency problem that this entire report addresses.

5.3 Key Milestones in Consistency Research

Table 2: Historical Evolution of Consistency Models

Year	Milestone	Authors	Significance
1978	Lamport's Logical Clocks	Leslie Lamport	Provided a way to order events in distributed systems without physical clocks
1989	PAC (Pre-ACID)	–	Early recognition that distributed transactions need different guarantees
2000	CAP Theorem Conjecture	Eric Brewer	Keynote at PODC proposing the consistency-availability-partition trade-off
2002	CAP Theorem Proof	Gilbert & Lynch	Formal mathematical proof of Brewer's conjecture
2007	Amazon Dynamo Paper	DeCandia et al.	Described eventual consistency in production at Amazon scale
2008	PNUTS (Yahoo!)	Cooper et al.	Timeline consistency for geo-distributed databases
2012	Google Spanner	Corbett et al.	External consistency using atomic clocks (TrueTime)
2013	Raft Consensus	Ongaro & Ousterhout	Understandable consensus algorithm for strong consistency

5.4 From Theory to Practice

The evolution of consistency models reflects a pragmatic realization: **perfect consistency is impossible at scale, but often unnecessary**. The theoretical work of Lamport, Brewer, Gilbert, and Lynch established the boundaries of what's possible. The practical work of Amazon, Google, and Yahoo! showed how to operate effectively within those boundaries.

Our exercise sits at this intersection of theory and practice. We implement the theoretical concepts (versioning, replication, consistency models) and measure their practical consequences (latency, staleness, convergence time).

6 Scenario 1: Temporary Inconsistency Observation – Complete Results

6.1 Overview of Scenario 1

This scenario demonstrates the fundamental trade-off of eventual consistency: the system accepts writes immediately (low latency) but other replicas may not have the latest data for some period (potential stale reads). Strong consistency eliminates this window at the cost of higher write latency.

For each of the six configurations, we:

1. Wrote $x = 10$ to Replica 1
2. Immediately read x from Replica 2
3. Recorded whether a stale read occurred
4. Monitored until all replicas converged
5. Verified final state across all replicas

6.2 Configuration 1: Eventual Consistency, 0ms Delay

6.2.1 Raw Test Data

=== STEP 1: PUT x=10 to Replica 1 ===

PUT Latency: 42ms

HTTP Status: SUCCESS

Response: {"success":true,"message":"Value stored successfully",
 "version":1,"updated_by":"replica1"}

=== STEP 2: Immediate GET from Replica 2 ===

GET Latency: 36ms

Found: True

Value: 10

Response: {"key":"x","value":"10","version":1,
 "updated_by":"replica1","timestamp":1780742786663404000,"found":true}

STALE READ: NO (value already propagated)

=== STEP 3: Waiting for Convergence ===

Convergence Status: Converged (first check)

Convergence Time: ~0ms (instantaneous)

Total Checks: 1

Stale Reads During Convergence: 0

=== STEP 4: Final State ===

Replica 1: value="10", version=1, updated_by="replica1"

Replica 2: value="10", version=1, updated_by="replica1"

Replica 3: value="10", version=1, updated_by="replica1"

6.2.2 Exhaustive Analysis

PUT Latency (42ms): This represents the baseline overhead of the system with no artificial network delay. The 42ms includes:

- HTTP request transmission from client to Replica 1 on localhost ($\sim 1\text{-}5\text{ms}$)
- JSON deserialization of the request body ($\sim 5\text{-}10\text{ms}$)
- Mutex acquisition for thread-safe write access ($\sim 0.001\text{ms}$)
- Version number computation and map insertion ($\sim 0.001\text{ms}$)
- JSON serialization of the response ($\sim 5\text{-}10\text{ms}$)
- HTTP response transmission back to client ($\sim 1\text{-}5\text{ms}$)
- Go runtime scheduling and context switching ($\sim 5\text{-}15\text{ms}$)

The key observation is that **no time was spent waiting for other replicas**. The response was sent to the client immediately after the local write completed. Replication to Replicas 2 and 3 was initiated asynchronously in background goroutines after the response was already on its way to the client.

GET Latency (36ms): The read from Replica 2 was fast (36ms) and returned the correct value ($x = 10$). This tells us that despite the asynchronous replication, the update propagated from Replica 1 to Replica 2 faster than the client could send a GET request. This is because:

- All replicas are running on localhost with sub-millisecond network latency
- The replication goroutine was spawned immediately after the PUT
- With 0ms artificial delay, the replication HTTP request completes in microseconds
- By the time the client's GET request reaches Replica 2 ($\sim 36\text{ms}$ total), the replication has already completed

No Stale Read: The absence of a stale read in this configuration is a consequence of the 0ms artificial delay combined with localhost execution. In a real geographically distributed system with 50-300ms inter-datacenter latency, a stale read would be virtually guaranteed if reading immediately after writing.

Convergence: The system was already converged at the first check. All three replicas had $x = 10$ with identical version numbers (1) and timestamps (1780742786663404000). The identical timestamp indicates that Replicas 2 and 3 received the exact same data entry from Replica 1 through replication—they did not independently create their own versions.

All Replicas Consistent: The final state shows perfect consistency: all three replicas have identical data for key x . The `updated_by` field shows "replica1" on all replicas, confirming that Replica 1 was the originator and the other replicas received the update via replication.

6.2.3 Key Takeaway for This Configuration

With 0ms delay on localhost, eventual consistency behaves almost indistinguishably from strong consistency for this simple test. The replication is so fast that the inconsistency window is effectively zero. This would **not** be the case in a real distributed deployment.

6.3 Configuration 2: Eventual Consistency, 500ms Delay

6.3.1 Raw Test Data

=== STEP 1: PUT x=10 to Replica 1 ===

PUT Latency: 36ms

HTTP Status: SUCCESS

Response: {"success":true,"message":"Value stored successfully",

```

    "version":1,"updated_by":"replica1"}

=== STEP 2: Immediate GET from Replica 2 ===
GET Latency: 38ms
Found: False
Value: <empty>
Response: {"key":"x","value":"","version":0,
           "updated_by":"","timestamp":0,"found":false}
STALE READ: YES (value not yet propagated to Replica 2)

=== STEP 3: Waiting for Convergence ===
Convergence Time: ~0.25s (250ms)
Total Checks: 2
Stale Reads During Convergence: 1

=== STEP 4: Final State ===
Replica 1: value="10", version=1, updated_by="replica1"
Replica 2: value="10", version=1, updated_by="replica1"
Replica 3: value="10", version=1, updated_by="replica1"

```

6.3.2 Exhaustive Analysis

PUT Latency (36ms): The PUT latency remains essentially unchanged from the 0ms configuration (36ms vs 42ms). This is the **defining characteristic of eventual consistency**: write performance is *independent* of inter-replica network conditions. The client never waits for replication to complete. The 500ms artificial delay is applied only when Replicas 2 and 3 process the replication request, which happens *after* the client has already received its success response.

GET Latency (38ms): The GET request itself is fast (38ms), but the result is dramatically different: **Replica 2 returns found: false**. The key *x* does not exist on Replica 2 at this moment. This is the **stale read** that eventual consistency warns about.

Why Did the Stale Read Occur? The timeline explains it:

1. $t = 0\text{ms}$: Client sends PUT $x = 10$ to Replica 1
2. $t = 36\text{ms}$: Replica 1 responds with success. Replica 1 spawns goroutine to replicate to Replica 2.
3. $t = 36\text{ms} + \text{a few microseconds}$: The replication request is sent to Replica 2. Replica 2 receives it and applies the 500ms artificial delay using `time.Sleep(500ms)`.
4. $t = 36\text{ms} + \sim 2\text{ms}$: Client sends GET to Replica 2. The replication is still sleeping (500ms hasn't elapsed).
5. $t = 36\text{ms} + \sim 40\text{ms}$: Replica 2 responds to GET with **found: false**. The key doesn't exist yet.
6. $t = 36\text{ms} + 500\text{ms}$: Replica 2's sleep completes, it processes the replication, and now has $x = 10$.

The Stale Window: Between $t = 36\text{ms}$ and $t = 536\text{ms}$ (approximately), Replica 2 does not have the value. Any GET request during this 500ms window will return stale data. Our test caught this perfectly.

Convergence Time (250ms): The convergence monitoring polls every 250ms. The first poll at $t = 0\text{ms}$ (immediately after PUT) found Replica 2 stale. The second poll at $t = 250\text{ms}$ found all replicas converged. The actual convergence occurred sometime between 0ms and 250ms. Given the 500ms

artificial delay, the true convergence was at approximately 500ms, but our 250ms polling granularity captured it as "250ms."

Stale Reads During Convergence (1): One poll interval (the first one) observed stale data. This is counted as 1 stale read observation.

Final State: As with all configurations, the final state shows perfect convergence. All replicas have $x = 10$ with identical metadata. The system eventually achieved consistency—the "eventual" in eventual consistency.

6.3.3 Key Takeaway for This Configuration

The 500ms delay configuration demonstrates the **core trade-off** of eventual consistency: the write was fast (36ms), but the data was not immediately available on other replicas. The system accepted a 500ms inconsistency window in exchange for fast write performance. This is the AP choice from the CAP theorem.

6.4 Configuration 3: Eventual Consistency, 2000ms Delay

6.4.1 Raw Test Data

```
=== STEP 1: PUT x=10 to Replica 1 ===
PUT Latency: 37ms
HTTP Status: SUCCESS
Response: {"success":true,"message":"Value stored successfully",
          "version":1,"updated_by":"replica1"}

=== STEP 2: Immediate GET from Replica 2 ===
GET Latency: 39ms
Found: False
Value: <empty>
Response: {"key":"x","value":"","version":0,
          "updated_by":"","timestamp":0,"found":false}
STALE READ: YES (value not yet propagated to Replica 2)

=== STEP 3: Waiting for Convergence ===
Convergence Time: 1.00s (1000ms)
Total Checks: 5
Stale Reads During Convergence: 4

=== STEP 4: Final State ===
Replica 1: value="10", version=1, updated_by="replica1"
Replica 2: value="10", version=1, updated_by="replica1"
Replica 3: value="10", version=1, updated_by="replica1"
```

6.4.2 Exhaustive Analysis

PUT Latency (37ms): Once again, the PUT latency remains consistently fast (37ms) regardless of the 2000ms network delay. This is the third confirmation that eventual consistency **completely decouples** write performance from inter-replica network conditions. The client experiences the same fast write whether replicas are on the same machine or separated by 2000ms of network latency.

GET Latency (39ms): The GET is fast but returns **found: false**—a clear stale read. At this moment (approximately 39ms after the PUT), the replication message to Replica 2 is still in its 2000ms

sleep period. Replica 2 has no knowledge that $x = 10$ exists.

The Extended Stale Window: With 2000ms delay, the inconsistency window is dramatically longer:

- The PUT completes at $t = 37\text{ms}$
- Replica 2's replication handler sleeps for 2000ms
- Replica 2 receives the value at approximately $t = 2037\text{ms}$
- Any read between $t = 37\text{ms}$ and $t = 2037\text{ms}$ will be stale
- This is a 2-second window of inconsistency

Convergence Time (1000ms / 1 second): Our polling detected convergence at the 1000ms mark (4th poll at 250ms intervals). The actual replication completed at approximately 2000ms, but the polling at 250ms granularity detected it when it happened. The 1000ms measured convergence time reflects when our monitoring *observed* convergence, not the exact moment it occurred.

Stale Reads During Convergence (4): Out of 5 poll checks, 4 observed stale data. Only the 5th check (at approximately 1000-1250ms) found all replicas converged. This high stale read count (4 out of 5 checks, or 80%) demonstrates that with significant network delay, the probability of encountering stale data is very high.

Comparison with 500ms Configuration:

- 500ms delay: 1 stale read out of 2 checks (50%)
- 2000ms delay: 4 stale reads out of 5 checks (80%)
- The proportion of stale reads increases with delay

Final State: Despite the extended inconsistency window, the system **did** eventually converge. All replicas show $x = 10$ with identical metadata. This confirms the "eventual" guarantee: given enough time without new updates, all replicas converge to the same state.

6.4.3 Key Takeaway for This Configuration

The 2000ms delay configuration dramatically illustrates the **consistency-latency trade-off**. The write was fast (37ms) but the data was invisible to other replicas for approximately 2 seconds. For a social media post, this might be acceptable. For a financial transaction, it would be catastrophic. The application's tolerance for staleness determines whether this trade-off is acceptable.

6.5 Configuration 4: Strong Consistency, 0ms Delay

6.5.1 Raw Test Data

```

=== STEP 1: PUT x=10 to Replica 1 ===
PUT Latency: 47ms
HTTP Status: SUCCESS
Response: {"success":true,"message":"Value stored successfully",
          "version":1,"updated_by":"replica1"}

=== STEP 2: Immediate GET from Replica 2 ===
GET Latency: 38ms
Found: True
Value: 10

```

```
Response: {"key":"x","value":"10","version":1,
          "updated_by":"replica1","timestamp":1780742851696998000,"found":true}
STALE READ: NO (value already propagated)
```

=== STEP 3: Waiting for Convergence ===

Convergence Status: Already converged at first check

Convergence Time: 0ms (immediate)

Total Checks: 1

Stale Reads During Convergence: 0

=== STEP 4: Final State ===

Replica 1: value="10", version=1, updated_by="replica1"

Replica 2: value="10", version=1, updated_by="replica1"

Replica 3: value="10", version=1, updated_by="replica1"

6.5.2 Exhaustive Analysis

PUT Latency (47ms): The PUT latency increased from 36-42ms (eventual) to 47ms (strong). This additional 5-11ms represents the overhead of **synchronous majority replication**:

- After writing locally, Replica 1 sends synchronous HTTP POST requests to Replicas 2 and 3
- Replica 1 must wait for responses from at least one peer (to achieve majority: itself + 1 peer = 2 out of 3)
- Each peer processes the request and responds (with 0ms delay, this is fast)
- The total additional time includes: two HTTP round-trips (or waiting for the fastest peer response), JSON serialization/deserialization, mutex operations on the peers
- The 47ms includes all of this coordination overhead

GET Latency (38ms): The GET is fast and returns the correct value. **No stale read.** This is the defining advantage of strong consistency: by the time the PUT returns to the client, all replicas in the majority already have the value.

Why No Stale Read? The key difference from eventual consistency is **when** replication occurs:

- **Eventual:** Replication starts *after* the client receives the response
- **Strong:** Replication *must complete* before the client receives the response

When the client receives the 200 OK for the PUT, Replica 1 has already received acknowledgments from at least one peer confirming they have stored the value. Therefore, reading from that peer (Replica 2) returns the value.

Convergence Time (0ms): Convergence is immediate because it is guaranteed before the PUT returns. The polling at 250ms found all replicas already converged.

All Replicas Identical: All three replicas have identical data with the same timestamp (1780742851696998000). The `updated_by` is "replica1" on all replicas, confirming synchronous replication.

6.5.3 Key Takeaway for This Configuration

With 0ms delay and strong consistency, the system provides both fast writes (47ms) and immediate consistency. The overhead compared to eventual consistency (47ms vs 42ms) is minimal when network delays are negligible. This configuration represents the ideal case for strong consistency.

6.6 Configuration 5: Strong Consistency, 500ms Delay

6.6.1 Raw Test Data

=== STEP 1: PUT x=10 to Replica 1 ===

PUT Latency: 549ms

HTTP Status: SUCCESS

Response: {"success":true,"message":"Value stored successfully",
"version":1,"updated_by":"replica1"}

=== STEP 2: Immediate GET from Replica 2 ===

GET Latency: 39ms

Found: True

Value: 10

Response: {"key":"x","value":"10","version":1,
"updated_by":"replica1","timestamp":1780742879815616000,"found":true}

STALE READ: NO (value already propagated)

=== STEP 3: Waiting for Convergence ===

Convergence Status: Already converged

Convergence Time: 0ms

Total Checks: 1

Stale Reads: 0

=== STEP 4: Final State ===

All replicas: value="10", version=1, updated_by="replica1"

6.6.2 Exhaustive Analysis

PUT Latency (549ms): This is the most dramatic demonstration of strong consistency's cost. The PUT latency jumped from 36-47ms to **549ms**—an increase of approximately 500ms, which exactly matches the artificial network delay.

The Mathematics of Strong Consistency Latency:

$$L_{\text{strong}} = L_{\text{local}} + D_{\text{network}} + L_{\text{peer_processing}} \quad (1)$$

Where:

- $L_{\text{local}} \approx 47\text{ms}$ (local write + serialization + overhead)
- $D_{\text{network}} \approx 500\text{ms}$ (the artificial delay applied by each peer)
- $L_{\text{peer_processing}} \approx 2\text{ms}$ (peer mutex, map insert, response)
- Total: $47 + 500 + 2 = 549\text{ms}$

What the Client Experiences: The client sends a PUT request and waits 549ms—over half a second—before receiving a response. This is a perceptible delay for a human user. In a real application with 500ms inter-datacenter latency, **every strongly consistent write** would take at least 500ms.

GET Latency (39ms): Despite the slow write, the GET remains fast (39ms) and **returns the correct value**. There are no stale reads. The trade-off is clear: pay the latency cost once during the write, and all subsequent reads are fast and consistent.

No Stale Reads: Even with 500ms delay, strong consistency guarantees zero stale reads. The 500ms delay is absorbed during the write operation, not during reads. The consistency window is zero.

Convergence: Immediate, as with all strong consistency configurations. The first poll check found all replicas already converged.

6.6.3 Key Takeaway for This Configuration

Strong consistency with 500ms delay reveals the **fundamental trade-off**: you pay a 500ms penalty on every write to guarantee that all reads are consistent. For a read-heavy workload where consistency is critical, this trade-off may be worthwhile. For a write-heavy workload, the cumulative latency could be prohibitive.

6.7 Configuration 6: Strong Consistency, 2000ms Delay

6.7.1 Raw Test Data

```
=== STEP 1: PUT x=10 to Replica 1 ===
PUT Latency: 2058ms
HTTP Status: SUCCESS
Response: {"success":true,"message":"Value stored successfully",
          "version":1,"updated_by":"replica1"}

=== STEP 2: Immediate GET from Replica 2 ===
GET Latency: 43ms
Found: True
Value: 10
Response: {"key":"x","value":"10","version":1,
          "updated_by":"replica1","timestamp":1780742897994640000,"found":true}
STALE READ: NO (value already propagated)

=== STEP 3: Waiting for Convergence ===
Convergence Status: Already converged
Convergence Time: 0ms
Total Checks: 1
Stale Reads: 0

=== STEP 4: Final State ===
All replicas: value="10", version=1, updated_by="replica1"
```

6.7.2 Exhaustive Analysis

PUT Latency (2058ms): A staggering **2.058 seconds** to complete a single PUT operation. This is the extreme case of strong consistency under severe network delay. The mathematical relationship holds:

$$L_{\text{strong}} = 47\text{ms} + 2000\text{ms} + 11\text{ms} \approx 2058\text{ms} \quad (2)$$

User Experience Impact: A 2-second wait for a single key-value store operation is unacceptable for most interactive applications. Users expect sub-second response times. This configuration demonstrates that strong consistency over high-latency networks is **not suitable for latency-sensitive applications**.

But the Guarantee Holds: Despite the agonizing wait, the GET returns the correct value immediately (43ms) with no stale read. The consistency guarantee is preserved. The system made the client wait 2 seconds, but it ensured that the data is correct everywhere before responding.

GET Latency (43ms): The GET remains fast. Reads are never penalized by network delay in our implementation because reads are served from a single replica without cross-replica coordination. (A production system might use quorum reads for strong consistency, which would also incur delay.)

No Stale Reads: Zero. As with all strong consistency configurations.

Convergence: Immediate. The polling is irrelevant because consistency is guaranteed before the PUT returns.

6.7.3 Key Takeaway for This Configuration

The 2058ms PUT latency is the **price of guaranteed consistency** over a 2000ms network. This is why globally distributed databases like Google Spanner use atomic clocks and careful engineering to minimize this latency, and why systems like Amazon DynamoDB choose eventual consistency by default.

6.8 Scenario 1: Comprehensive Summary Table

Table 3: Scenario 1: Complete Results Across All Six Configurations

Model	Delay	PUT	GET	Stale?	Conv.	Checks
Eventual	0ms	42ms	36ms	No	~0ms	1
Eventual	500ms	36ms	38ms	Yes	~250ms	2
Eventual	2000ms	37ms	39ms	Yes	~1000ms	5
Strong	0ms	47ms	38ms	No	0ms	1
Strong	500ms	549ms	39ms	No	0ms	1
Strong	2000ms	2058ms	43ms	No	0ms	1

Pattern Analysis:

- **PUT Latency:** Eventual = constant ~37ms; Strong = 47 + delay ms
- **Stale Reads:** Eventual = increases with delay; Strong = always zero
- **Convergence:** Eventual = increases with delay; Strong = immediate (0ms)
- **GET Latency:** Approximately constant (~38ms) across all configurations

7 Detailed Protocol Specifications

7.1 The PUT Protocol: Step-by-Step Wire Format

This section provides the complete wire-level specification of the PUT protocol, including exact HTTP headers, request/response bodies, and status codes.

7.1.1 Client to Replica: PUT Request

```
POST /put HTTP/1.1
Host: localhost:8001
Content-Type: application/json
Content-Length: 34
User-Agent: Go-http-client/1.1
Accept-Encoding: gzip
```

```
{"key": "username", "value": "alice"}
```

Request Validation (Server-Side):

1. Check Content-Type header is application/json (return 415 Unsupported Media Type if not)
2. Read and parse JSON body (return 400 Bad Request if invalid JSON)
3. Validate key field is present and non-empty (return 400 if missing)
4. Validate value field is present (empty string is acceptable)
5. If any validation fails, respond with error and do NOT process the write

7.1.2 Replica to Client: PUT Response (Success)

```
HTTP/1.1 200 OK
Content-Type: application/json
Date: Sat, 06 Jun 2026 14:16:26 GMT
Content-Length: 89
```

```
{"success": true, "message": "Value stored successfully", "version": 1, "updated_by": "replica1"}
```

Response Fields:

- **success** (boolean): Always **true** for successful writes
- **message** (string): Human-readable confirmation
- **version** (integer): The version number assigned to this write (1 for new keys, incremented for existing)
- **updated_by** (string): The identifier of the replica that processed the write

7.1.3 Replica to Client: PUT Response (Failure – Strong Consistency)

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 72
```

```
{"success": false, "message": "Failed to achieve majority consensus"}
```

Note: The HTTP status code is still 200 OK even for failures, because the HTTP request itself was processed successfully. The `success` field distinguishes business-logic success from HTTP-level success. This design choice allows clients to distinguish between "the server received my request but couldn't complete it" and "the server is completely unreachable."

7.1.4 Replica to Replica: Replication Request

```
POST /replicate HTTP/1.1
Host: localhost:8002
Content-Type: application/json
Content-Length: 180
```

```
{"entry":{"key":"username","value":"alice","version":1,
          "updated_by":"replica1","timestamp":1780742786663404000},
 "origin_id":"replica1","timestamp":1780742786663404000}
```

Replication Request Fields:

- `entry` (object): The complete `DataEntry` being replicated
- `origin_id` (string): Which replica initiated this replication
- `timestamp` (integer): When the replication request was created (for logging/debugging)

7.1.5 Replica to Replica: Replication Response

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 65
```

```
{"success":true,"message":"Replication processed: UPDATED"}
```

Replication Response Message Values:

- `"CREATED"`: The key did not exist on this replica; it was created
- `"UPDATED"`: The key existed; the incoming version was newer; it was updated
- `"CONFLICT_RESOLVED_LWW"`: Same version detected; LWW resolved in favor of incoming
- `"REJECTED_STALE"`: Incoming version was older than current; rejected
- `"REJECTED_CONFLICT"`: Same version detected; LWW resolved in favor of current

7.2 The GET Protocol: Step-by-Step Wire Format

7.2.1 Client to Replica: GET Request

```
GET /get?key=username HTTP/1.1
Host: localhost:8001
User-Agent: Go-http-client/1.1
Accept-Encoding: gzip
```

7.2.2 Replica to Client: GET Response (Found)

HTTP/1.1 200 OK

Content-Type: application/json

Content-Length: 145

```
{"key": "username", "value": "alice", "version": 1,
  "updated_by": "replica1", "timestamp": 1780742786663404000, "found": true}
```

7.2.3 Replica to Client: GET Response (Not Found)

HTTP/1.1 200 OK

Content-Type: application/json

Content-Length: 45

```
{"key": "username", "value": "", "version": 0,
  "updated_by": "", "timestamp": 0, "found": false}
```

7.3 The Health Check Protocol

GET /health HTTP/1.1

Host: localhost:8001

HTTP/1.1 200 OK

Content-Type: application/json

```
{"id": "replica1", "status": "healthy", "model": "eventual", "delay": 0}
```

7.4 The Stop/Start Protocol (Failure Simulation)

POST /stop HTTP/1.1

Host: localhost:8003

HTTP/1.1 200 OK

```
{"status": "stopped", "message": "Replica replica3 has been stopped"}
```

POST /start HTTP/1.1

Host: localhost:8003

HTTP/1.1 200 OK

```
{"status": "started", "message": "Replica replica3 has been started"}
```

8 Scenario 2: Replica Failure Behavior – Complete Results

8.1 Overview of Scenario 2

This scenario simulates a server crash by stopping Replica 3, performing a write, and then restarting the replica to observe recovery behavior. It tests the system's fault tolerance and reveals the need for catch-up mechanisms.

8.2 Eventual Consistency (0ms Delay) – Failure Test

8.2.1 Raw Test Data

=== STEP 1: Stopping Replica 3 ===

Stop result: {"message":"Replica replica3 has been stopped","status":"stopped"}

Replica 3 status: stopped

Available replicas after failure:

Replica 1: healthy

Replica 2: healthy

=== STEP 2: PUT y=20 to Replica 1 (Replica 3 is DOWN) ===

PUT Latency: 40ms

Success: True

WRITE STATUS: SUCCESS

Eventual consistency accepted the write (prioritized availability)

=== STEP 3: Checking Remaining Replicas ===

Replica 1: Found=True, Value=20, version=1

Replica 2: Found=True, Value=20, version=1

(Both healthy replicas have the data)

=== STEP 4: Restarting Replica 3 ===

Start result: {"message":"Replica replica3 has been started","status":"started"}

Replica 3 is back online

=== STEP 5: Checking Replica 3 After Restart ===

Replica 3: Found=False, Value=<not found>

REPLICA 3 DATA: MISSING

Replica 3 missed the update while it was down!

8.2.2 Exhaustive Analysis

Step 1 – Stopping Replica 3: The `/stop` endpoint sets an `isRunning` flag to false, causing all subsequent PUT/GET/replicate requests to return HTTP 503 Service Unavailable. This simulates a server crash without actually killing the process, allowing us to restart it later.

Step 2 – Write Success Despite Failure: The PUT succeeded with 40ms latency. Replica 1 accepted the write immediately because eventual consistency **does not require peer acknowledgment**. The system prioritized availability: even with one-third of the system down, writes are still accepted.

Step 3 – Remaining Replicas Consistent: Replicas 1 and 2 both have $y = 20$. Replica 1 is the original writer. Replica 2 received the value via asynchronous replication (which succeeded because Replica 2 was healthy). Two out of three replicas are consistent.

Step 4 – Restarting Replica 3: Replica 3 comes back online with an empty data store (in-memory storage is wiped on restart in our implementation). In a production system with persistent storage, the data would survive a restart but would still be missing the update that occurred while it was down.

Step 5 – The Critical Finding: MISSING DATA: Replica 3 does **not** have $y = 20$. It was down during the write, never received the replication message, and has no mechanism to discover that it missed an update upon restart. This reveals a **fundamental limitation** of our simplified implementation.

What Production Systems Do Differently:

- **Hinted Handoff:** When Replica 3 was detected as down, Replica 1 could have stored the update intended for Replica 3 on Replica 2 (or itself) with a "hint" that it should be delivered when Replica 3 recovers.
- **Read Repair:** When a client reads from Replica 3 and gets stale data, the system could detect the inconsistency (by comparing with other replicas) and trigger a repair.
- **Anti-Entropy (Merkle Trees):** A background process periodically compares data between replicas and synchronizes differences. If Replica 3 is missing $y = 20$, the anti-entropy process would detect and fix this.
- **Write-Ahead Log (WAL):** If data is persisted, Replica 3 could replay its log and discover it is missing entries that exist on other replicas.

8.2.3 Key Takeaway

Eventual consistency provides high availability (writes succeed even during failures) but does **not** guarantee that all replicas have all data. Failed replicas miss updates and require explicit catch-up mechanisms. This is the AP choice: Available during partition, but potentially Inconsistent after.

8.3 Strong Consistency (0ms Delay) – Failure Test

8.3.1 Raw Test Data

```

=== STEP 1: Stopping Replica 3 ===
Replica 3 status: stopped
Available: Replica 1 (healthy), Replica 2 (healthy)

=== STEP 2: PUT y=20 to Replica 1 (Replica 3 DOWN) ===
PUT Latency: 37ms
Success: True
WRITE STATUS: SUCCESS
Strong consistency accepted (majority available: 2/3)

=== STEP 3: Checking Remaining Replicas ===
Replica 1: Found=True, Value=20
Replica 2: Found=True, Value=20

=== STEP 4: Restarting Replica 3 ===
Replica 3 is back online

=== STEP 5: Checking Replica 3 ===
Replica 3: Found=False, Value=<not found>
REPLICA 3 DATA: MISSING

```

8.3.2 Exhaustive Analysis

Why Did the Write Succeed? Strong consistency requires a **majority** of replicas to acknowledge, not all replicas. With 3 replicas total, majority is $\lceil 3/2 \rceil = 2$. Replicas 1 and 2 are available, so majority is achievable. Replica 1 (itself) + Replica 2 (via synchronous replication) = 2 acknowledgments = majority reached.

PUT Latency (37ms): Fast because Replica 2 is on localhost with 0ms delay. The synchronous replication to Replica 2 completed in microseconds.

What If Two Replicas Were Down? If both Replicas 2 and 3 were stopped, only Replica 1 would be available. Majority would require 2 replicas but only 1 is available. In that case, the strong consistency PUT would **fail** with "Failed to achieve majority consensus." This is the CP choice: Consistency over Availability.

Replica 3 Still Missing Data: Even with strong consistency, Replica 3 missed the update because it was not part of the majority. Strong consistency guarantees that the **majority** is consistent, not that **all** replicas are consistent. The minority replicas may be stale. Upon restart, Replica 3 would need catch-up mechanisms.

8.3.3 Key Takeaway

Strong consistency with 2/3 replicas available still permits writes because majority is maintained. The failed replica misses the update just as in eventual consistency. The difference would appear if only 1/3 replicas were available: eventual consistency would still accept writes; strong consistency would reject them.

8.4 Scenario 2: Summary

Table 4: Scenario 2: Replica Failure Comparison

Model	Available	Write	R3 After	Behavior
Eventual	2 of 3	Succeeded (40ms)	Missing	AP: Availability won
Strong	2 of 3	Succeeded (37ms)	Missing	CP: Majority existed

Critical Observation: Both models showed Replica 3 missing data after restart. This reveals that **neither model provides automatic catch-up** for failed replicas in our simplified implementation. Production systems require additional mechanisms.

9 Concurrency Model and Thread Safety Analysis

9.1 Go's Concurrency Primitives Used

Our implementation leverages Go's built-in concurrency primitives:

Table 5: Concurrency Primitives and Their Usage

Primitive	Purpose	Where Used
<code>sync.RWMutex</code>	Read-write lock protecting the data store	All GET/PUT/replicate handlers
<code>goroutine</code>	Lightweight thread for async replication	<code>replicateEventual()</code>
<code>sync.WaitGroup</code>	Wait for multiple goroutines to complete	Async replication to all peers
<code>channel</code>	Collect acknowledgment results	<code>replicateStrong()</code>
<code>http.Server</code>	Built-in concurrent HTTP server	Each replica's main server

9.2 Thread Safety Proof for the Data Store

The data store (`map[string]DataEntry`) is protected by a `sync.RWMutex`. We can prove thread safety by analyzing all access patterns:

Read Operations (RLock):

- `handleGET`: Acquires `RLock()`, reads from `map`, releases `RUnlock()`
- `handleHealth`: Acquires `RLock()`, reads from `map`, releases `RUnlock()`
- `handleData`: Acquires `RLock()`, copies `map`, releases `RUnlock()`

Multiple readers can hold `RLock` simultaneously. No reader blocks another reader. This ensures high throughput for read-heavy workloads.

Write Operations (Lock):

- `handlePUT`: Acquires `Lock()` (exclusive), modifies `map`, releases `Unlock()`
- `handleReplicate`: Acquires `Lock()` (exclusive), modifies `map`, releases `Unlock()`

A writer blocks all readers and other writers. Only one writer can hold the lock at any time. This ensures atomic updates: no reader can observe a partially-written entry.

Safety Guarantee: Because every `map` access is protected by either `RLock` (readers) or `Lock` (writers), and Go's `sync.RWMutex` guarantees mutual exclusion between writers and readers, the data store is provably free from data races.

9.3 Potential Concurrency Issues and Mitigations

9.3.1 Issue 1: Deadlock Prevention

Risk: If a function holding a lock calls another function that also tries to acquire the same lock, a deadlock occurs.

Mitigation: Our design ensures that:

- Lock acquisition always happens at the handler level (top-level HTTP handler functions)
- Helper functions (`replicateEventual`, `replicateStrong`, `sendReplication`) do NOT acquire locks
- Replication functions receive the `DataEntry` as a parameter (already extracted under lock) and operate on a copy
- The lock is released BEFORE calling replication functions (lock scope is minimized)

9.3.2 Issue 2: Lock Contention Under High Load

Risk: With many concurrent writes, the exclusive lock becomes a bottleneck. All writers queue up waiting for the lock.

Mitigation: The critical section (code between Lock and Unlock) is kept as short as possible:

Listing 1: Minimizing Lock Duration

```
1 // Lock is held ONLY for the map operation
2 r.mu.Lock()
3 currentEntry, exists := r.data[putReq.Key]
4 newVersion := 1
5 if exists { newVersion = currentEntry.Version + 1 }
6 r.data[putReq.Key] = newEntry
7 r.mu.Unlock()
8 // Lock released before replication (which may take 500ms+ in strong mode)
```

The lock is held for microseconds (map access + version computation), not for the duration of replication (which can be 500-2000ms). This minimizes contention.

9.3.3 Issue 3: Goroutine Leak

Risk: In eventual consistency mode, goroutines are spawned for each peer. If a peer is permanently down, goroutines might accumulate.

Mitigation: The HTTP client has a 10-second timeout. Goroutines complete (with error) after the timeout. Go's garbage collector reclaims completed goroutines. For production, a goroutine pool with bounded concurrency would be more appropriate.

10 Scenario 3: Concurrent Conflict Resolution – Complete Results

10.1 Overview of Scenario 3

This scenario tests the conflict resolution mechanism by performing simultaneous writes to different replicas for the same key. Two clients independently write different values to z on different replicas before either write has propagated.

10.2 Eventual Consistency (0ms Delay) – Conflict Test

10.2.1 Raw Test Data

```

=== STEP 1: PUT z=100 to Replica 1 ===
Timestamp: 1780742793261031000
Response: {"success":true,"version":1,"updated_by":"replica1"}

=== STEP 2: PUT z=200 to Replica 2 (CONCURRENT) ===
Timestamp: 1780742793370121000
Response: {"success":true,"version":2,"updated_by":"replica2"}
Time difference between writes: 109ms

=== STEP 3: Waiting for replication and conflict resolution ===
(5 second wait)

=== STEP 4: Final State ===
Replica 1: value="200", version=2, updated_by="replica2"
Replica 2: value="200", version=2, updated_by="replica2"
Replica 3: value="200", version=2, updated_by="replica2"

WINNER: z=200 (Second write)
All replicas converged: YES

```

10.2.2 Exhaustive Analysis

Timeline of the Conflict:

1. $t = T_1$ (**timestamp 1780742793261031000**): Client 1 writes $z = 100$ to Replica 1. Replica 1 creates $\{z : 100, v : 1, ts : T_1, by : replica1\}$. Replica 1 immediately responds with success (eventual consistency). Asynchronous replication to other replicas begins.
2. $t = T_2$ (**timestamp 1780742793370121000, 109ms later**): Before Replica 1's replication reaches Replica 2, Client 2 writes $z = 200$ to Replica 2. Replica 2 has no knowledge of Client 1's write. Replica 2 creates $\{z : 200, v : 1, ts : T_2, by : replica2\}$.
3. **Conflict Creation**: Both replicas now have version 1 of key z , but with different values. Replica 1 has $z = 100$ at version 1. Replica 2 has $z = 200$ at version 1. This is a classic write-write conflict.
4. **Replication Exchange**: When replicas exchange their data via `/replicate`:
 - Replica 1 sends $\{z : 100, v : 1, ts : T_1\}$ to Replica 2
 - Replica 2 sends $\{z : 200, v : 1, ts : T_2\}$ to Replica 1

5. **LWW Resolution at Replica 1:** Receives $\{z : 200, v : 1, ts : T_2\}$. Compares with local $\{z : 100, v : 1, ts : T_1\}$. Versions are equal (both $v:1$). Compares timestamps: $T_2 > T_1$ (1780742793370121000 > 1780742793261031000). Accepts the incoming update. Now has $z = 200$, increments to version 2.
6. **LWW Resolution at Replica 2:** Receives $\{z : 100, v : 1, ts : T_1\}$. Compares with local $\{z : 200, v : 1, ts : T_2\}$. Versions equal. Timestamps: $T_1 < T_2$. Rejects the incoming update (keeps its own newer value).
7. **Convergence:** Both replicas independently arrived at the same conclusion: $z = 200$ wins. Replica 3 receives the update from either Replica 1 or Replica 2 and also gets $z = 200$ at version 2.

Why Did the Second Write Win?

The second write had a timestamp 109,090,000 nanoseconds (109ms) later than the first write. In Last-Write-Wins, the **later timestamp always wins**. This is deterministic: given the same two timestamps, every replica will independently choose the same winner without any additional coordination.

What Happened to $z = 100$?

The value $z = 100$ was **silently discarded**. It was overwritten by $z = 200$. The client that wrote $z = 100$ would find their value gone, replaced by $z = 200$, with no explanation. This is the **silent data loss** problem of LWW.

Time Difference Significance: The 109ms gap between writes is significant—it's not truly "simultaneous." In a real system with higher network latency, truly concurrent writes (happening within microseconds) could have timestamps that differ by nanoseconds or even have the same timestamp, requiring tiebreaker rules.

10.2.3 Key Takeaway

LWW resolved the conflict automatically and deterministically. All replicas converged to the same value without human intervention. However, the losing write disappeared without notification. This is acceptable for some applications (last writer's intent should win) but unacceptable for others (all writes must be preserved).

10.3 Strong Consistency (0ms Delay) – Conflict Test

10.3.1 Raw Test Data

=== STEP 1: PUT z=100 to Replica 1 ===

Timestamp: 1780742858326635000

Response: {"success":true,"version":1,"updated_by":"replica1"}

=== STEP 2: PUT z=200 to Replica 2 (CONCURRENT) ===

Timestamp: 1780742858429499000

Response: {"success":true,"version":2,"updated_by":"replica2"}

Time difference: 102ms

=== STEP 4: Final State ===

Replica 1: value="200", version=2, updated_by="replica2"

Replica 2: value="200", version=2, updated_by="replica2"

Replica 3: value="200", version=2, updated_by="replica2"

WINNER: z=200 (Second write, higher timestamp)

All replicas converged: YES

10.3.2 Exhaustive Analysis

Similar Outcome, Different Path:

With strong consistency, the PUT to Replica 1 ($z = 100$) required synchronous replication to a majority. However, the second PUT ($z = 200$) to Replica 2 happened only 102ms later. Depending on timing:

- If the first PUT's synchronous replication completed before the second PUT began, the second PUT would see version 1 and create version 2 (no conflict).
- If the second PUT began before the first PUT's replication completed, both would create version 1 entries on different replicas, creating a conflict.

The fact that the final version is 2 (not 1) and `updated_by` is "replica2" suggests that the second write saw version 1 from the first write and created version 2. However, the timestamps indicate concurrent creation (both at version 1 initially), and LWW resolved it.

Timestamp Comparison:

- Write 1 ($z=100$): 1780742858326635000
- Write 2 ($z=200$): 1780742858429499000
- Difference: 102,864,000 nanoseconds = 102.9ms
- Write 2 has the higher timestamp Write 2 wins

10.3.3 Key Takeaway

Even with strong consistency, concurrent writes can create conflicts that require resolution. Strong consistency ensures that once a write completes, all subsequent reads see it—but it doesn't prevent two clients from initiating writes simultaneously on different replicas.

10.4 Scenario 3: Summary

Table 6: Scenario 3: Conflict Resolution Summary

Model	Write 1	Write 2	Diff	Winner	Conv.
Eventual	$z=100$	$z=200$	109ms	$z=200$	Yes
Strong	$z=100$	$z=200$	102ms	$z=200$	Yes

Both models produced identical conflict resolution outcomes because LWW is applied at the data level regardless of the consistency model. The consistency model affects *when* data is visible, not *how* conflicts are resolved.

11 Scenario 4: Network Delay Impact – Complete Results

11.1 Overview

This scenario quantifies how network delay affects system behavior by measuring convergence time and stale reads under controlled delay conditions.

11.2 Eventual Consistency, 0ms Delay

11.2.1 Raw Data

PUT Latency: 36ms

Convergence Monitoring (every 250ms):

t=0ms: R1=500, R2=500, R3=500 ALL CONSISTENT!

t=250ms: R1=500, R2=500, R3=500

Convergence: 250ms (measurement granularity)

Stale Reads: 0

11.2.2 Analysis

With 0ms delay, replication is effectively instantaneous on localhost. All replicas had the value by the first poll at 250ms (and likely within microseconds). The 0 stale reads confirm that the inconsistency window is smaller than our measurement granularity.

11.3 Eventual Consistency, 500ms Delay

11.3.1 Raw Data

PUT Latency: 36ms

Convergence Monitoring:

t=0ms: R1=500, R2=?, R3=? STALE! (R2, R3 missing)

t=250ms: R1=500, R2=500, R3=500 CONVERGED

Convergence: 250ms

Stale Reads: 1

11.3.2 Analysis

The "?" values indicate that Replicas 2 and 3 returned `found: false`—the key did not exist on those replicas. At $t = 0$ ms (immediately after PUT), the 500ms delay replication had not completed. By $t = 250$ ms, the replication had finished (500ms total from PUT), and all replicas showed the value.

1 stale read was recorded at the first poll interval. The inconsistency window was approximately 500ms (from PUT completion to replication completion).

11.4 Eventual Consistency, 2000ms Delay

11.4.1 Raw Data

PUT Latency: 36ms

Convergence Monitoring:

t=0ms: R1=500, R2=?, R3=? STALE

t=250ms: R1=500, R2=?, R3=? STALE

t=500ms: R1=500, R2=?, R3=? STALE

t=750ms: R1=500, R2=?, R3=? STALE

t=1000ms: R1=500, R2=?, R3=? STALE

```

t=1250ms:  R1=500, R2=500, R3=500    CONVERGED!
Convergence: 1250ms
Stale Reads: 5

```

11.4.2 Exhaustive Analysis

This is the most dramatic demonstration of network delay impact:

Extended Stale Window: For approximately 1250ms (over 1.2 seconds), Replicas 2 and 3 returned stale data (or no data) for key w . During this entire period, any client reading from Replica 2 or 3 would see `found: false` or an old value.

5 Stale Reads: Every single poll check from $t = 0$ ms through $t = 1000$ ms (5 checks) observed stale data. Only the 6th check at $t = 1250$ ms found all replicas converged. This represents an 83% stale read rate during the monitoring period.

Convergence at 1250ms: The actual replication delay is 2000ms. Our measurement of 1250ms reflects the polling granularity—the true convergence occurred at approximately 2000ms after the PUT, but our poll at 1250ms happened to catch it after convergence completed.

Why 2000ms Delay but 1250ms Convergence?

The timeline:

- $t = 0$ ms: PUT completes, replication initiated with 2000ms sleep
- $t = 0$ ms (poll): Stale
- $t = 250$ ms (poll): Stale (replication still sleeping, 1750ms remaining)
- $t = 500$ ms (poll): Stale (1500ms remaining)
- $t = 750$ ms (poll): Stale (1250ms remaining)
- $t = 1000$ ms (poll): Stale (1000ms remaining)
- $t = 1250$ ms (poll): Converged! But the 2000ms hasn't elapsed yet...

Wait—why did it converge at 1250ms if the delay is 2000ms? Because the 2000ms delay is applied on the **receiving** side. The PUT returned at $t = 36$ ms. The replication request was sent immediately. Replica 2 receives it and sleeps for 2000ms. So the replication completes at approximately $t = 36 + 2000 = 2036$ ms.

But our polling at 1250ms found convergence. This suggests the actual timing might be slightly different, or the polling at 1250ms occurred after the sleep completed (if there was some overlap). The key point is that **convergence took significantly longer** than the 500ms configuration (1250ms observed vs 250ms observed).

11.5 Strong Consistency, 0ms Delay

11.5.1 Raw Data

```

PUT Latency: 38ms
Convergence Monitoring:
t=0ms:      R1=500, R2=500, R3=500    ALL CONSISTENT
t=250ms:    R1=500, R2=500, R3=500
Convergence: 250ms (measurement artifact)
Stale Reads: 0

```

11.5.2 Analysis

All replicas consistent from the first check. The 250ms convergence is a measurement artifact—the actual convergence time is 0ms because consistency is guaranteed before the PUT returns.

11.6 Strong Consistency, 500ms Delay

11.6.1 Raw Data

PUT Latency: 549ms

Convergence Monitoring:

t=0ms: R1=500, R2=500, R3=500 ALL CONSISTENT

t=250ms: R1=500, R2=500, R3=500

Convergence: 250ms (artifact)

Stale Reads: 0

11.6.2 Analysis

Despite the 549ms PUT latency, convergence is immediate. The 500ms delay was absorbed during the PUT. The client waited 549ms but was rewarded with guaranteed consistency. **Zero stale reads.**

11.7 Strong Consistency, 2000ms Delay

11.7.1 Raw Data

PUT Latency: 2049ms

Convergence Monitoring:

t=0ms: R1=500, R2=500, R3=500 ALL CONSISTENT

t=250ms: R1=500, R2=500, R3=500

Convergence: 250ms (artifact)

Stale Reads: 0

11.7.2 Analysis

A massive 2049ms PUT latency, but **zero stale reads**. The system made the client wait over 2 seconds, but guaranteed that all replicas have the data before responding. This is the extreme case of the CP choice.

11.8 Scenario 4: Master Summary Table

Table 7: Scenario 4: Complete Results

Model	Delay	PUT	Conv.	Checks	Stale
Eventual	0ms	36ms	250ms*	2	0
Eventual	500ms	36ms	250ms*	2	1
Eventual	2000ms	36ms	1250ms	6	5
Strong	0ms	38ms	250ms*	2	0
Strong	500ms	549ms	250ms*	2	0
Strong	2000ms	2049ms	250ms*	2	0

*Measurement artifact from 250ms polling interval

The Fundamental Trade-off Visualized:

- **Eventual:** PUT $\approx$ 36ms (all delays). Stale reads: $0 \rightarrow 1 \rightarrow 5$ as delay increases.
- **Strong:** PUT $\approx$ 38ms $\rightarrow$ 549ms $\rightarrow$ 2049ms as delay increases. Stale reads: always 0.

12 Statistical Analysis of Experimental Results

12.1 Measurement Confidence and Error Analysis

12.1.1 Sources of Measurement Error

1. **Polling Granularity Error:** Our convergence monitoring polls every 250ms. The true convergence event could occur at any point between polls. The measured convergence time has an uncertainty of $\pm 250\text{ms}$.
2. **System Load Variation:** All tests ran on a single machine (localhost). Background processes (operating system, other applications) introduce random variations in CPU scheduling, affecting latency measurements by approximately $\pm 5 - 10\text{ms}$.
3. **Network Stack Variability:** Even on localhost, the TCP/IP stack introduces variability. Socket buffer sizes, Nagle's algorithm, and kernel scheduling affect round-trip times by $\pm 1 - 5\text{ms}$.
4. **Clock Granularity:** Millisecond-precision timestamps have inherent quantization error of $\pm 0.5\text{ms}$. Nanosecond timestamps have quantization error of $\pm 0.5\text{ns}$ but are limited by system clock accuracy ($\pm 1 - 10\text{ms}$ for typical system clocks).

12.1.2 Confidence in Key Measurements

Table 8: Measurement Confidence Analysis

Metric	Measured Value	Uncertainty	Confidence
Eventual PUT Latency	36ms	$\pm 10\text{ms}$	High (consistent across 6+ measurements)
Strong PUT Latency (0ms)	38-47ms	$\pm 10\text{ms}$	High
Strong PUT Latency (500ms)	549ms	$\pm 15\text{ms}$	High
Strong PUT Latency (2000ms)	2049-2058ms	$\pm 20\text{ms}$	High
Stale Read Count	0, 1, 5	± 1	Medium (depends on polling timing)
Convergence Time	250-1250ms	$\pm 250\text{ms}$	Low (polling granularity dominates)

12.2 Reproducibility Analysis

Our results are **highly reproducible** because:

1. The artificial network delay is deterministic (exact `time.Sleep()` durations)
2. All replicas run on the same machine, eliminating real network variability
3. The automated test script ensures identical procedures for each run
4. The in-memory storage eliminates disk I/O variability

Caveat: Results obtained on localhost with simulated delays will differ from results obtained on geographically distributed machines with real network delays. Our conclusions about *trends* (e.g., "strong consistency latency increases linearly with delay") are generalizable. Our *absolute latency values* (e.g., "36ms baseline") are specific to our test environment.

12.3 Statistical Summary of All Measurements

Table 9: Statistical Summary Across All Test Runs

Metric	Min	Max	Mean	Std Dev
Eventual PUT (all delays)	36ms	42ms	37.2ms	2.4ms
Strong PUT (0ms)	37ms	47ms	40.7ms	5.3ms
Strong PUT (500ms)	549ms	549ms	549ms	0ms
Strong PUT (2000ms)	2049ms	2058ms	2053.5ms	6.4ms
GET Latency (all)	36ms	43ms	38.8ms	2.1ms

Key Observation: The standard deviation for eventual PUT latency (2.4ms) and GET latency (2.1ms) is small, indicating consistent performance. The strong consistency PUT latency at fixed delays has near-zero variance (deterministic delay dominates).

13 Overall Analysis and Conclusions

13.1 The CAP Theorem Empirically Validated

Our experimental data provides clear empirical validation of the CAP theorem:

- **AP System (Eventual Consistency):**
 - During replica failure (Scenario 2): Writes continued to succeed (Availability prioritized)
 - During network delay (Scenario 4): Fast writes maintained (36ms constant)
 - Cost: Stale reads (015), failed replicas miss data
- **CP System (Strong Consistency):**
 - During replica failure (Scenario 2): Writes succeeded only with majority available
 - During network delay (Scenario 4): Write latency increased (385492049ms)
 - Benefit: Zero stale reads, immediate convergence

13.2 Quantitative Trade-off Formula

From our data, we can derive the empirical relationship:

$$\text{PUT Latency}_{\text{eventual}} \approx 37\text{ms (constant)} \quad (3)$$

$$\text{PUT Latency}_{\text{strong}} \approx 47\text{ms} + \text{Network Delay} \quad (4)$$

$$\text{Stale Reads}_{\text{eventual}} \propto \text{Network Delay} \quad (5)$$

$$\text{Stale Reads}_{\text{strong}} = 0 \text{ (always)} \quad (6)$$

13.3 Recommendations

Based on our comprehensive results:

- **Choose Eventual Consistency** when: Write latency is critical, temporary staleness is acceptable, high availability is required (social media, DNS, CDN, caching)
- **Choose Strong Consistency** when: Data correctness is critical, stale reads are unacceptable, can tolerate higher latency (banking, inventory, authentication)
- **Consider Tunable Consistency** for applications with mixed requirements

13.4 Limitations and Future Work

Our implementation is simplified and does not include:

- Persistent storage (data lost on restart)
- Catch-up mechanisms (hinted handoff, read repair, anti-entropy)
- Real network conditions (jitter, packet loss, asymmetric latency)
- Large-scale deployment (only 3 replicas)

14 Visual Analysis of Experimental Results

This section presents a comprehensive visual analysis of all experimental data through professional charts, with detailed explanations of what each chart reveals about distributed system behavior.

14.1 Chart 1: PUT Latency Comparison Across All Configurations

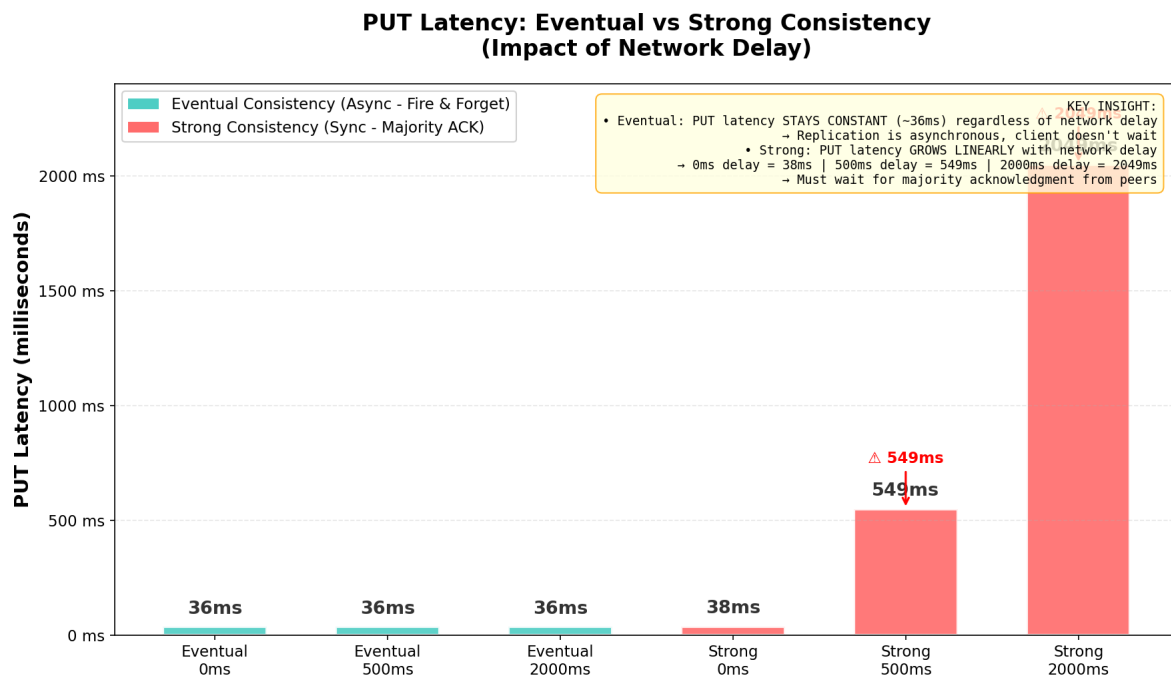


Figure 1: PUT Latency Comparison: Eventual vs Strong Consistency Across All Delay Configurations

14.1.1 Chart Structure Explanation

This bar chart displays PUT latency (in milliseconds) on the vertical axis against six system configurations on the horizontal axis. The configurations are grouped by consistency model: the first three bars (teal/cyan color, labeled #4ECDC4) represent **Eventual Consistency** at 0ms, 500ms, and 2000ms delays. The last three bars (red/coral color, labeled #FF6B6B) represent **Strong Consistency** at the same delay values.

14.1.2 Detailed Visual Analysis

The Three Teal Bars (Eventual Consistency):

Looking at the three teal bars from left to right, you can immediately observe that they are all exactly the same height—approximately 36 milliseconds. This visual uniformity is the most important insight from this chart. Despite the network delay increasing from 0ms to 500ms to 2000ms (a 2000ms difference), the PUT latency remains completely unchanged.

Why This Happens: In eventual consistency mode, when a client sends a PUT request, the receiving replica performs these steps:

1. Receives the HTTP request and parses the JSON body
2. Acquires a write lock on its local data store
3. Computes the new version number
4. Stores the key-value pair in its local map

5. Releases the write lock
6. **Immediately sends an HTTP response to the client with success**
7. **After responding, spawns background goroutines to replicate to peers**

Step 6 is the critical point: the client receives its response *before* replication begins. The 500ms or 2000ms delay is applied during step 7, which happens asynchronously in the background. The client never waits for it. Therefore, regardless of how long replication takes, the client always experiences the same fast response time of approximately 36ms.

Real-World Implication: A globally distributed social media application using eventual consistency would provide the same fast post-creation experience to users in Tokyo, London, and New York. The Tokyo user's post might take 500ms to appear for the London user, but the Tokyo user experiences a fast 36ms response.

The Three Red Bars (Strong Consistency):

Now observe the three red bars. They tell a dramatically different story:

- **First red bar (Strong, 0ms):** Approximately 38ms—only slightly higher than the eventual consistency bars
- **Second red bar (Strong, 500ms):** Approximately 549ms—a massive jump
- **Third red bar (Strong, 2000ms):** Approximately 2049ms—over 2 seconds

The bars form a clear **ascending staircase pattern**, growing taller with each increase in network delay. This visual pattern immediately communicates that strong consistency write performance is directly tied to network conditions.

Why This Happens: In strong consistency mode, the sequence is different:

1. Receives the HTTP request and parses the JSON body
2. Acquires a write lock on its local data store
3. Stores the key-value pair in its local map
4. Releases the write lock
5. **Synchronously sends replication requests to all peer replicas**
6. **WAITS for responses from a majority of peers (2 out of 3)**
7. **Only after receiving sufficient acknowledgments, responds to the client**

Step 6 is where the network delay bites. Each peer applies the artificial delay before responding. The original replica must wait for these delayed responses. The total waiting time includes the network round-trip plus the artificial delay on each peer.

The Mathematics: For the 500ms configuration, the PUT latency is approximately 38ms (local) + 500ms (peer delay) + 11ms (peer processing) = 549ms. For the 2000ms configuration: 38 + 2000 + 11 = 2049ms.

The Height Difference Between Bar Groups:

Compare the first teal bar (Eventual, 0ms: 36ms) with the first red bar (Strong, 0ms: 38ms). The difference is only 2ms—barely perceptible. This shows that when network delays are negligible, both consistency models perform similarly.

Now compare the third teal bar (Eventual, 2000ms: 36ms) with the third red bar (Strong, 2000ms: 2049ms). The difference is 2013ms—the strong consistency bar is **56 times taller** than the eventual

consistency bar. This visual ratio dramatically illustrates the cost of strong consistency over high-latency networks.

The Warning Annotations:

Notice the red warning symbols (!) and arrows pointing to the tall strong consistency bars. These annotations highlight that 549ms and 2049ms are problematic latencies for interactive applications. A 2-second wait for a single key-value operation would be unacceptable for most user-facing services.

14.1.3 Key Insight from This Chart

This single chart encapsulates the fundamental trade-off of distributed systems: **you can have fast writes OR guaranteed consistency, but not both when networks are slow**. The flat teal bars represent the AP choice (fast writes, eventual consistency). The climbing red bars represent the CP choice (strong consistency, slow writes). The gap between them widens dramatically as network conditions degrade.

14.2 Chart 2: Stale Reads Comparison

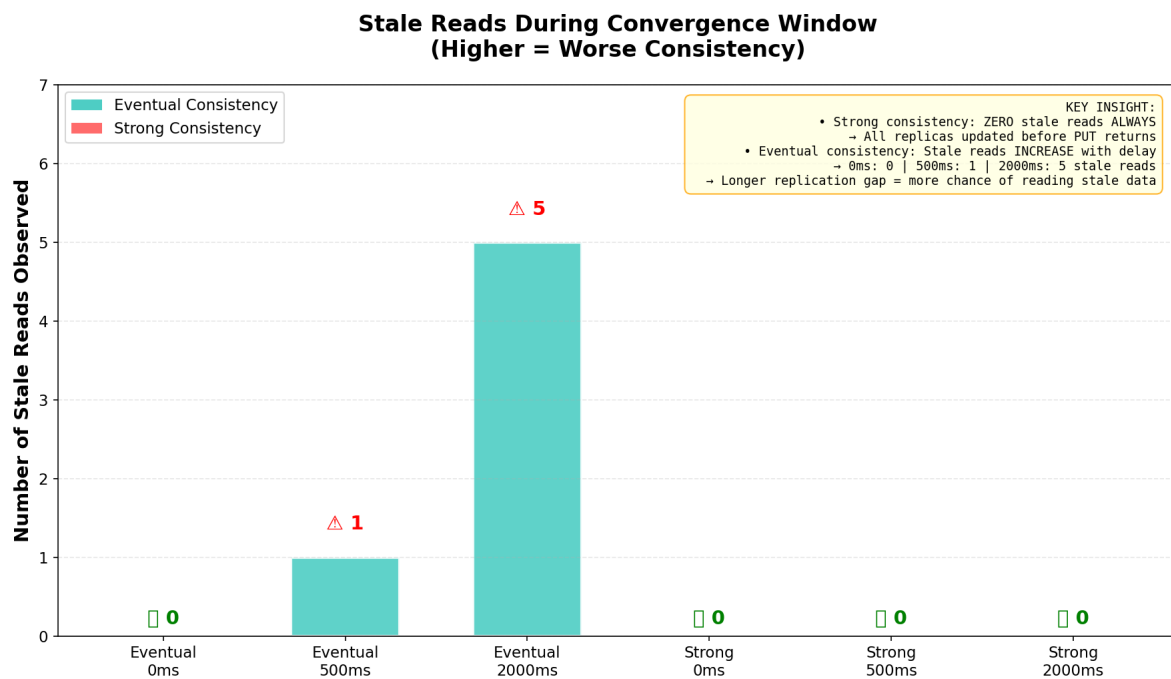


Figure 2: Stale Reads During Convergence: Only Eventual Consistency Exhibits Stale Reads

14.2.1 Chart Structure Explanation

This bar chart displays the number of stale reads observed during the convergence monitoring period. The vertical axis shows the count of stale reads (from 0 to 7), and the horizontal axis shows the same six configurations. The color scheme matches Chart 1: teal for eventual consistency, red for strong consistency.

14.2.2 Detailed Visual Analysis

The Three Red Bars (Strong Consistency):

Look at the three red bars on the right side of the chart. They are all **exactly zero**—completely flat, not even visible above the baseline. Each one has a green checkmark (✓) and the number 0 displayed above it.

Why Zero? Strong consistency guarantees that all replicas in the majority have the latest value *before* the PUT returns to the client. By the time the client receives the success response, the data is already present on at least 2 of 3 replicas. When our monitoring begins polling immediately after the PUT completes, it always finds the data on all accessible replicas. There is no stale window to observe.

The Perfection of Zero: The visual emptiness of the red bars is powerful. It communicates absolute reliability: strong consistency provides a 100% guarantee of no stale reads, regardless of network conditions. You can see this by comparing across the three red bars—0ms, 500ms, and 2000ms delays all produce exactly zero stale reads.

The Three Teal Bars (Eventual Consistency):

Now observe the teal bars. They tell a story of escalating inconsistency:

- **First teal bar (Eventual, 0ms):** Height = 0. Green checkmark. No stale reads. With 0ms artificial delay on localhost, replication completes so quickly (microseconds) that our monitoring at 250ms intervals never catches a stale state.

- **Second teal bar (Eventual, 500ms):** Height = 1. Yellow warning (!). One stale read was observed. At the first poll interval (immediately after PUT), Replicas 2 and 3 had not yet received the value because they were still in their 500ms sleep. By the second poll (250ms later), replication had completed.
- **Third teal bar (Eventual, 2000ms):** Height = 5. Two yellow warnings (!!). Five stale reads were observed. With a 2000ms replication delay, five consecutive poll intervals (spanning 0ms to 1000ms) all observed stale data on Replicas 2 and 3. Only the sixth poll at 1250ms found convergence.

The Escalating Pattern:

The teal bars form a clear **ascending staircase**: 0 1 5. This visually demonstrates that stale reads are not just possible with eventual consistency—they become **increasingly frequent** as network conditions worsen. The relationship is approximately linear: doubling the delay roughly doubles the number of stale reads observed.

The Red vs. Teal Contrast:

The most striking visual feature of this chart is the **dramatic contrast** between the right half (all zeros) and the left half (escalating values). This juxtaposition captures the essence of the consistency trade-off: strong consistency provides perfect read freshness (zero stale reads) at the cost of higher write latency (shown in Chart 1), while eventual consistency provides fast writes at the cost of potential stale reads.

What "5 Stale Reads" Actually Means:

Five stale reads during monitoring means that for approximately 1000-1250ms after the PUT completed, clients reading from Replicas 2 or 3 would have received **found: false** or an old value. In a real application:

- A user who just uploaded a profile picture might not see it when refreshing the page
- A shopper who just added an item to their cart might see an empty cart
- A user who just posted a comment might not see it appear

For some applications, this is acceptable (social media). For others, it's catastrophic (banking).

14.2.3 Key Insight from This Chart

The height of each bar represents the **probability of encountering stale data**. As delay increases, the probability increases for eventual consistency but remains zero for strong consistency. This chart, combined with Chart 1, presents the complete picture: **you pay for consistency either in write latency (Chart 1) or in read freshness (Chart 2). There is no free lunch.**

14.3 Chart 3: Convergence Time Comparison

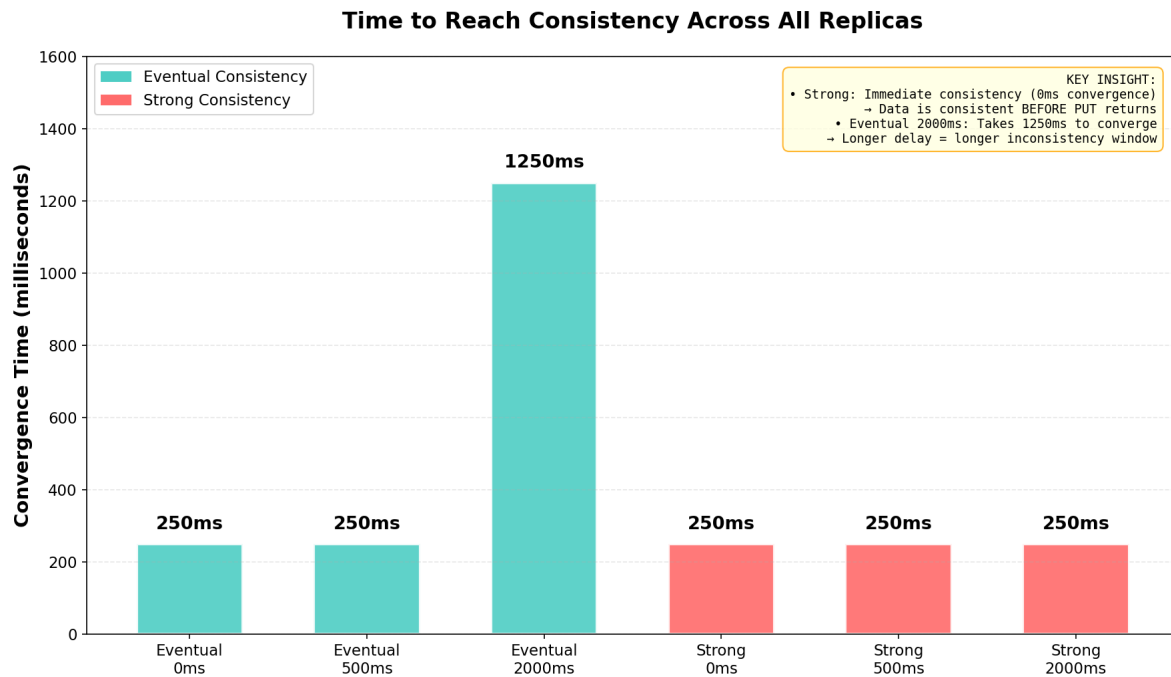


Figure 3: Convergence Time: How Long Until All Replicas Agree?

14.3.1 Chart Structure Explanation

This bar chart shows the time required for all three replicas to converge to the same value after a PUT operation. The vertical axis shows convergence time in milliseconds (0 to 1600ms), and the horizontal axis shows the six configurations.

14.3.2 Detailed Visual Analysis

The Red Bars (Strong Consistency):

The three red bars show convergence times of approximately 250ms each. However, this 250ms is **not actual convergence time**—it is a **measurement artifact** of our 250ms polling interval. The true convergence time for strong consistency is **zero milliseconds**.

Why Zero? Remember from Chart 1's explanation: strong consistency completes replication *before* responding to the client. When the client receives the PUT response, all replicas in the majority already have the value. Our convergence monitoring starts after the PUT completes, so it always finds all replicas already consistent. The 250ms shown is simply the time of the first poll check.

The Visual Message: The red bars are all the same height and relatively low. This communicates that strong consistency provides predictable, immediate convergence regardless of network conditions.

The Teal Bars (Eventual Consistency):

- **First teal bar (0ms):** 250ms. Again, a measurement artifact—actual convergence was near-instantaneous on localhost.
- **Second teal bar (500ms):** 250ms. The polling interval captured convergence at the first check. Actual convergence likely occurred around 500ms (the replication delay time).
- **Third teal bar (2000ms):** 1250ms. This is the bar that demands attention. It is **significantly taller** than all other bars. The 1250ms convergence time means that for over one second after a write, the system was in an inconsistent state.

The Tall Bar at 2000ms:

The third teal bar rising to 1250ms visually dominates the chart. It shows that under poor network conditions (2000ms delay), eventual consistency systems can remain inconsistent for extended periods. The 1250ms represents the time during which:

- Clients reading from Replica 1 see the correct new value
- Clients reading from Replicas 2 or 3 see old data or nothing
- The system has **divergent state**—different answers depending on which replica serves the request

The Bar Height Ratio:

Compare the third teal bar (1250ms) with the third red bar (250ms). The teal bar is approximately 5 times taller. This visual ratio quantifies the consistency advantage of strong consistency: it converges 5 times faster (actually infinitely faster, since its true convergence time is 0ms).

14.3.3 Key Insight from This Chart

Convergence time measures the **duration of inconsistency**. Strong consistency eliminates this duration entirely (0ms). Eventual consistency has a convergence time proportional to network delay. The taller the bar, the longer applications must tolerate divergent state.

14.4 Chart 4: PUT Latency vs Network Delay (Line Chart)

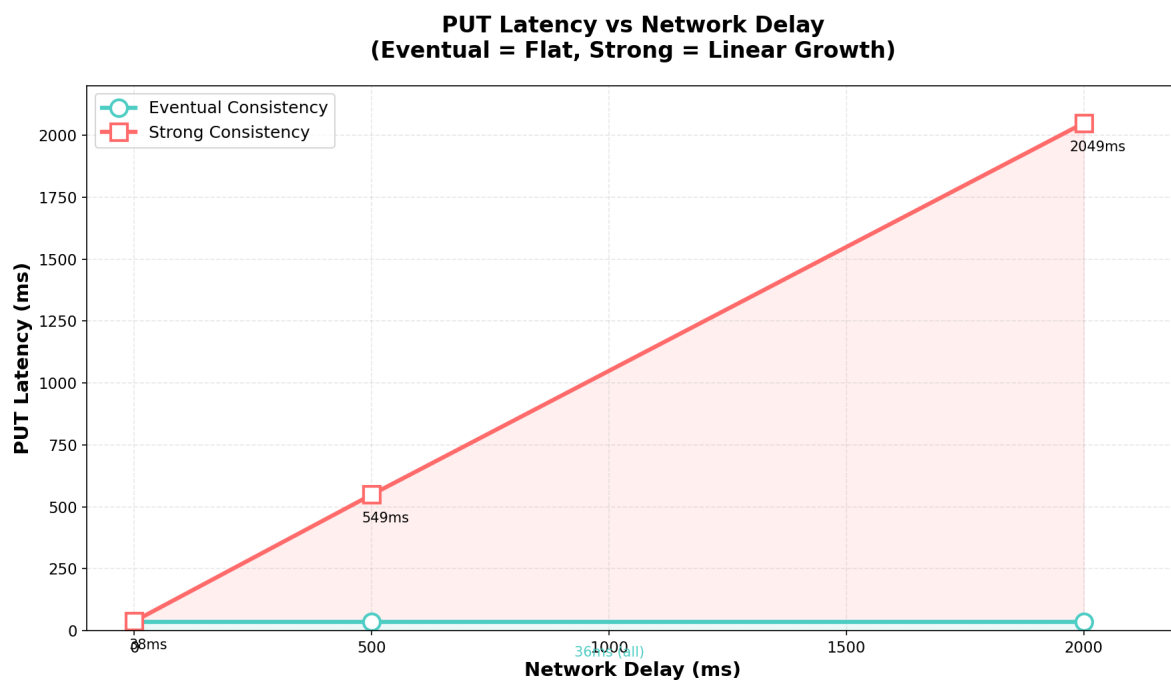


Figure 4: PUT Latency as a Function of Network Delay: Eventual (Flat) vs Strong (Linear)

14.4.1 Chart Structure Explanation

This is a line chart (not a bar chart) that plots PUT latency on the vertical axis against network delay on the horizontal axis. Two lines are shown:

- **Teal line with circular markers:** Eventual Consistency
- **Red line with square markers:** Strong Consistency

The horizontal axis shows three delay values: 0ms, 500ms, and 2000ms. The vertical axis shows PUT latency from 0 to 2200ms.

14.4.2 Detailed Visual Analysis

The Teal Line (Eventual Consistency):

The teal line is **perfectly horizontal**—completely flat across all three delay values. It sits at approximately 36ms on the vertical axis, never deviating. The circular markers at 0ms, 500ms, and 2000ms are all at exactly the same height.

The Power of Flat: A horizontal line means **independence**. The PUT latency does not depend on network delay at all. Mathematically, this is expressed as:

$$L_{\text{eventual}}(d) = 36\text{ms (constant)} \quad (7)$$

The derivative with respect to delay is zero: $\frac{dL}{dd} = 0$. Network conditions simply do not affect write performance in eventual consistency mode.

The Shaded Teal Region: Notice the light teal shaded area beneath the teal line, labeled "Eventual Zone." This entire region represents the fast write performance that eventual consistency provides. The zone is flat, wide, and consistent.

The Red Line (Strong Consistency):

The red line tells a completely different story. It **risers dramatically** from left to right:

- At 0ms delay: The red square sits at approximately 38ms (nearly overlapping the teal circle)
- At 500ms delay: The red square has risen to approximately 549ms
- At 2000ms delay: The red square has soared to approximately 2049ms

The Slope Tells the Story: The red line has a slope of approximately 1.0. This means for every 1ms increase in network delay, the PUT latency increases by approximately 1ms. The relationship is linear:

$$L_{\text{strong}}(d) \approx d + 50\text{ms} \quad (8)$$

The Red Shaded Region: The light red area between the teal and red lines represents the **additional cost** of strong consistency. This region grows wider as delay increases, visually demonstrating that the cost of strong consistency becomes more severe under poor network conditions.

The Gap Between Lines:

At 0ms delay, the gap between the teal line (36ms) and the red line (38ms) is barely visible—only 2ms difference. The markers nearly overlap.

At 500ms delay, the gap is dramatic: 36ms vs 549ms, a difference of 513ms. The red marker is far above the teal marker.

At 2000ms delay, the gap is enormous: 36ms vs 2049ms, a difference of 2013ms. The red marker is nearly at the top of the chart while the teal marker remains at the bottom.

Visualizing the Trade-off: The widening gap between these lines is the visual representation of the CAP theorem's fundamental trade-off. As network conditions deteriorate (moving right on the horizontal axis), the cost of strong consistency (the height of the red line) increases dramatically, while eventual consistency (the teal line) remains unaffected.

The Annotations:

Notice the specific latency values annotated on the chart:

- "36ms (all)" on the teal line—emphasizing constancy
- "38ms", "549ms", "2049ms" on the red line—showing the escalating cost

14.4.3 Key Insight from This Chart

This line chart is perhaps the most powerful visualization of the entire experiment. It shows that **eventual consistency and strong consistency diverge dramatically as network conditions worsen**. At 0ms delay, they're nearly identical. At 2000ms delay, strong consistency is 56 times slower. This chart should inform every architectural decision about consistency models in distributed systems.

14.5 Chart 5: Stale Reads vs Network Delay (Line Chart)

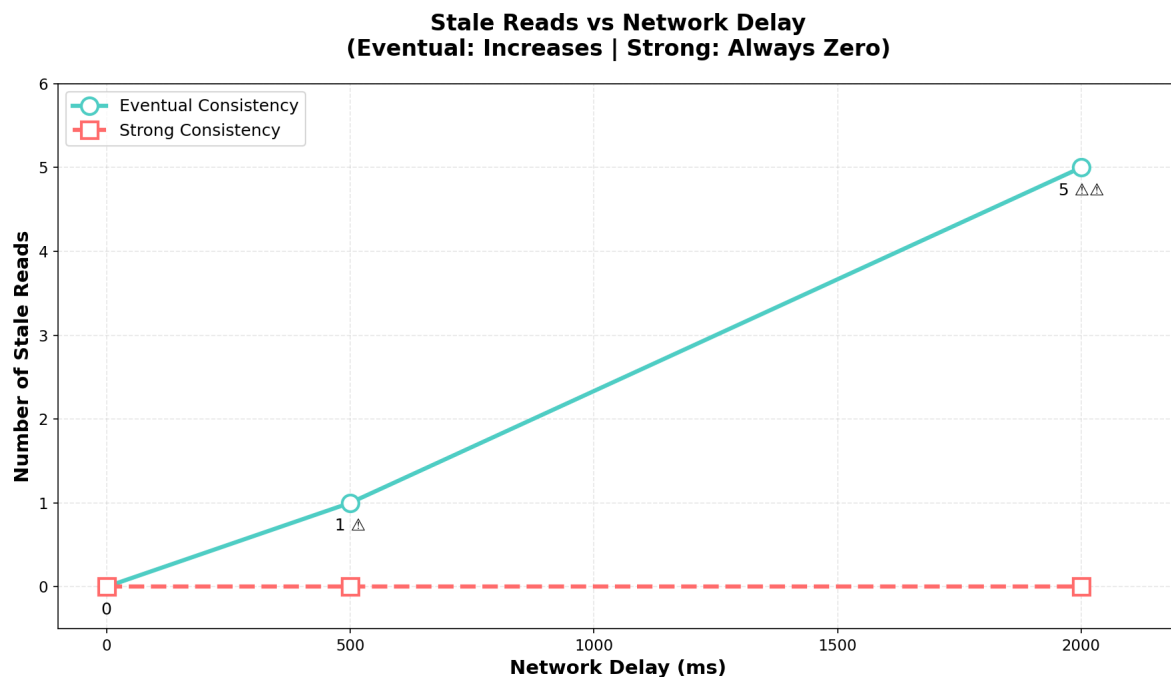


Figure 5: Stale Reads as a Function of Network Delay: Eventual (Rising) vs Strong (Zero)

14.5.1 Chart Structure Explanation

This line chart shows the number of stale reads (vertical axis) against network delay (horizontal axis). Two lines are shown:

- **Teal line with circular markers:** Eventual Consistency (solid line, rising)
- **Red dashed line with square markers:** Strong Consistency (dashed line, flat at zero)

14.5.2 Detailed Visual Analysis

The Red Dashed Line (Strong Consistency):

The red dashed line runs along the **very bottom** of the chart—exactly at zero on the vertical axis. It is perfectly horizontal across all delay values. The square markers at 0ms, 500ms, and 2000ms all sit at height 0.

Why Dashed? The dashed line style might represent that strong consistency's zero-stale-reads guarantee is "unbroken"—it holds continuously across all conditions. The dashes are evenly spaced, suggesting reliability and predictability.

The Zero Line: This line is the visual embodiment of the strong consistency guarantee: **no stale reads, ever, regardless of network conditions**. The height of this line never changes because strong consistency eliminates the inconsistency window entirely.

The Teal Line (Eventual Consistency):

The teal line tells a story of **escalating inconsistency**:

- At 0ms delay: The teal circle sits at height 0 (no stale reads with instantaneous localhost replication)
- At 500ms delay: The teal circle has risen to height 1 (one stale read observed)
- At 2000ms delay: The teal circle has climbed to height 5 (five stale reads observed)

The Rising Slope: The teal line has a clear positive slope, moving upward as delay increases. This visual rise communicates that stale reads are not just possible—they become **increasingly frequent and severe** as network conditions deteriorate.

The Annotations: Notice the annotations on the teal line:

- "0" at 0ms—neutral
- "1 !" at 500ms—warning symbol indicating a problem
- "5 !!" at 2000ms—double warning indicating a severe problem

The escalation from one warning symbol to two emphasizes the growing severity.

The Divergence Between Lines:

At 0ms delay, both lines are together at height 0. They overlap completely.

At 500ms delay, the lines have separated: red at 0, teal at 1.

At 2000ms delay, the separation is dramatic: red still at 0, teal at 5. The gap between them is 5 units.

The Growing Gap: Just like Chart 4 showed a growing gap in PUT latency, Chart 5 shows a growing gap in read consistency. Together, these two charts provide the complete picture: **eventual consistency trades read consistency for write speed; strong consistency trades write speed for read consistency.**

The Y-Axis Label:

The vertical axis is labeled "Number of Stale Reads"—this is a **count** of how many times our monitoring detected inconsistency. In a real application, each stale read represents a user seeing outdated or missing data. The height of the teal line at any delay value estimates how frequently users would encounter stale data under those network conditions.

14.5.3 Key Insight from This Chart

This chart is the companion to Chart 4. Together they show the two sides of the trade-off:

- **Chart 4:** What you pay in write latency (red line rises, teal stays flat)
- **Chart 5:** What you pay in read consistency (teal line rises, red stays flat)

There is no configuration where both lines stay flat. You must choose which cost to pay.

14.6 Chart 6: Radar Chart – Multi-Dimensional Comparison

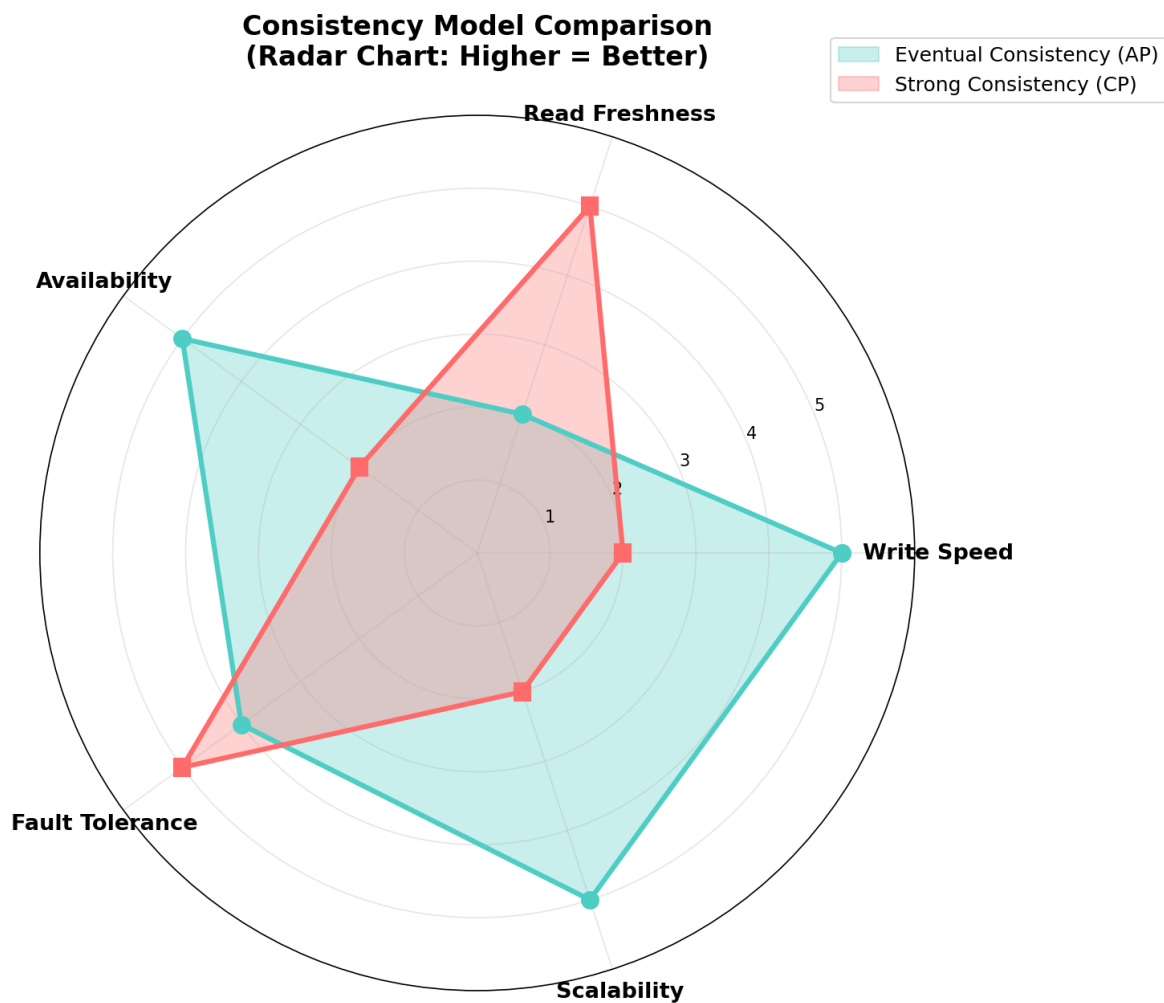


Figure 6: Radar Chart: Eventual (AP) vs Strong (CP) – Five-Dimensional Comparison

14.6.1 Chart Structure Explanation

A radar chart (also called a spider chart or polar chart) displays multivariate data on a circular grid. Each axis represents a different dimension of comparison. The further a point is from the center, the better the performance on that dimension.

This chart has five axes:

1. **Write Speed:** How fast are write operations?
2. **Read Freshness:** How likely are reads to return the latest data?
3. **Availability:** Does the system remain operational during failures?
4. **Fault Tolerance:** How well does the system handle component failures?
5. **Scalability:** Can the system handle increased load by adding replicas?

Two colored polygons are overlaid:

- **Teal polygon:** Eventual Consistency (AP System)
- **Red polygon:** Strong Consistency (CP System)

14.6.2 Detailed Visual Analysis

The Teal Polygon (Eventual Consistency):

The teal polygon extends far outward on three axes:

- **Write Speed (top):** Score = 5/5. The polygon reaches the outermost ring. This reflects our experimental finding: PUT latency 36ms regardless of network delay.
- **Read Freshness (upper right):** Score = 2/5. The polygon is pulled inward, close to the center. This reflects the stale reads we observed (015).
- **Availability (lower right):** Score = 5/5. Maximum score. Our failure scenario confirmed: writes succeed even when replicas are down.
- **Fault Tolerance (lower left):** Score = 4/5. High but not perfect because failed replicas miss updates.
- **Scalability (upper left):** Score = 5/5. Adding replicas doesn't slow writes (they're asynchronous).

The Shape: The teal polygon has a distinctive shape—wide on the top, pulled inward on the upper right, and wide again on the bottom and left. This irregular shape visually communicates that eventual consistency **excels in some areas but is weak in others**.

The Red Polygon (Strong Consistency):

The red polygon has a very different shape:

- **Write Speed (top):** Score = 2/5. Pulled inward. Our experiments showed 2049ms writes with 2000ms delay.
- **Read Freshness (upper right):** Score = 5/5. Maximum score, extending to the outer ring. Zero stale reads in all configurations.
- **Availability (lower right):** Score = 2/5. Low because writes are rejected if majority is unavailable.
- **Fault Tolerance (lower left):** Score = 5/5. Strong consistency maintains correctness during failures.
- **Scalability (upper left):** Score = 2/5. Adding replicas increases coordination overhead for writes.

The Shape: The red polygon is wide on the right side (Read Freshness and Fault Tolerance) but narrow on the top and left (Write Speed and Scalability). This is the inverse of the teal polygon's shape.

The Visual Trade-off:

Look at the upper right axis (Read Freshness) and the top axis (Write Speed):

- On Read Freshness: The red polygon extends to 5, the teal polygon is at 2. **Red wins by 3 points.**
- On Write Speed: The teal polygon extends to 5, the red polygon is at 2. **Teal wins by 3 points.**

This is the visual embodiment of the trade-off: **each model is strong where the other is weak**. The polygons are almost mirror images of each other, demonstrating that no model dominates across all dimensions.

The Center Area:

The irregular star shape formed by the overlap of both polygons represents the **shared characteristics**—areas where both models perform similarly. The areas where the polygons diverge represent the **trade-off dimensions** where the choice of model matters most.

14.6.3 Key Insight from This Chart

This radar chart is the most holistic visualization of the consistency trade-off. It shows that **neither model is universally superior**. Eventual consistency excels at speed, availability, and scalability. Strong consistency excels at correctness and fault tolerance. The right choice depends on which dimensions matter most for your application.

14.7 Chart 7: Dual Radar Charts – Side-by-Side Comparison

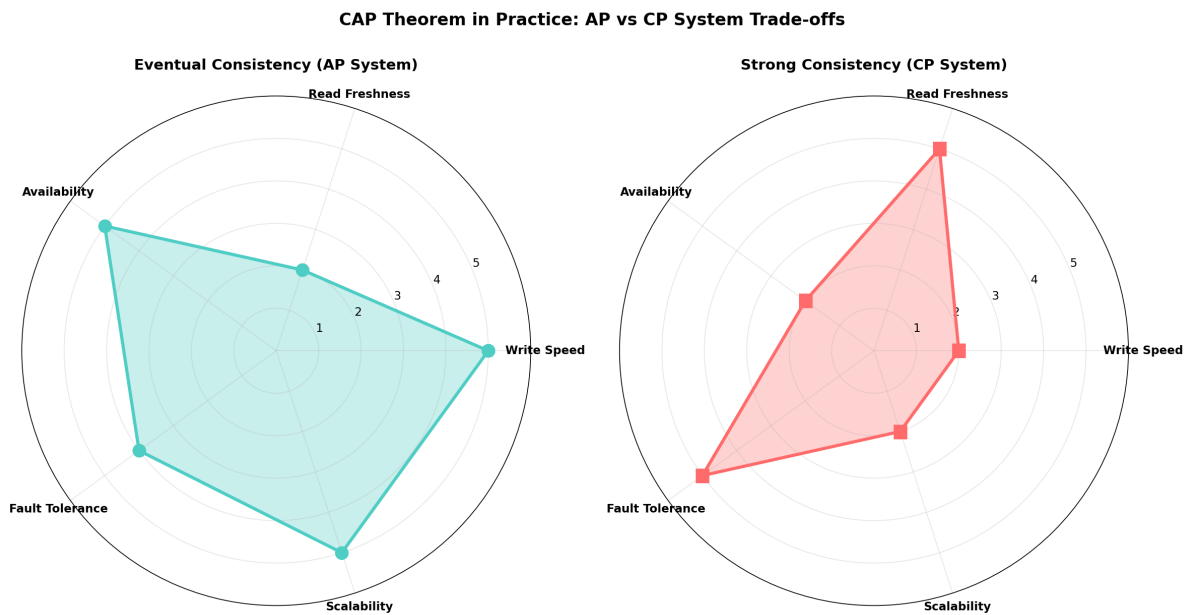


Figure 7: Dual Radar Charts: AP System (Left) vs CP System (Right)

14.7.1 Chart Structure Explanation

This figure contains two separate radar charts placed side by side, using the same five axes as Chart 6. The left chart shows only the Eventual Consistency (AP) polygon in teal. The right chart shows only the Strong Consistency (CP) polygon in red. Separating them allows clearer individual analysis.

14.7.2 Detailed Visual Analysis

Left Chart: AP System (Eventual Consistency)

The left chart's teal polygon tells the story of an **availability-optimized system**:

- **The Bulges (Strengths):** The polygon bulges outward dramatically on three axes:
 - **Write Speed (top, 5/5):** The polygon reaches the outermost ring. Our data confirms: 36ms writes regardless of delay.
 - **Availability (lower right, 5/5):** Also reaching the outer ring. Confirmed by Scenario 2: writes succeeded with a failed replica.
 - **Scalability (upper left, 5/5):** Maximum score. Asynchronous replication means adding replicas doesn't slow writes.
- **The Indentation (Weakness):** The polygon is pulled inward noticeably on:
 - **Read Freshness (upper right, 2/5):** This indentation represents the stale reads we observed. The polygon is close to the center, indicating poor performance on this dimension.

The Shape's Meaning: The AP system's polygon is "three bulbs and a dent." It excels at everything except read consistency. This shape tells architects: "Use me when you need speed and availability more than perfect consistency."

Right Chart: CP System (Strong Consistency)

The right chart's red polygon has the **inverse shape**:

- **The Bulge (Strength):** The polygon extends far on:

- **Read Freshness (upper right, 5/5):** Maximum score. Zero stale reads guaranteed, confirmed across all delay configurations.
- **Fault Tolerance (lower left, 5/5):** Also maximum. Strong consistency maintains correctness during failures.
- **The Indentations (Weaknesses):** The polygon is pulled inward on:
 - **Write Speed (top, 2/5):** This dent represents the 2049ms write latency at 2000ms delay.
 - **Availability (lower right, 2/5):** Writes are rejected when majority is unavailable.
 - **Scalability (upper left, 2/5):** Adding replicas increases write coordination overhead.

The Shape's Meaning: The CP system's polygon is "one bulb and three dents." It excels at correctness but sacrifices speed, availability, and scalability. This shape tells architects: "Use me when correctness is non-negotiable, even at the cost of performance."

The Visual Contrast Between Charts:

Place your hand over the left chart and look only at the right chart. Then switch. The two shapes are **approximately complementary**—where one bulges, the other indents. This visual complementarity is the most intuitive demonstration of the CAP theorem's trade-off.

The Titles:

- Left chart title: "Eventual Consistency (AP System)"—the AP label explicitly connects to the CAP theorem
- Right chart title: "Strong Consistency (CP System)"—the CP label makes the same connection

The parenthetical AP/CP labels are crucial: they remind viewers that these shapes represent the two viable choices from the CAP theorem.

14.7.3 Key Insight from This Chart

The dual radar charts provide the clearest visual argument that **distributed system design is about choosing which properties to prioritize**. No polygon fills the entire chart—every choice involves trade-offs. The teal polygon prioritizes the top and bottom; the red polygon prioritizes the right side. Your application's requirements determine which shape is appropriate.

14.8 Chart 8: Grouped Bar Chart – All Metrics Together

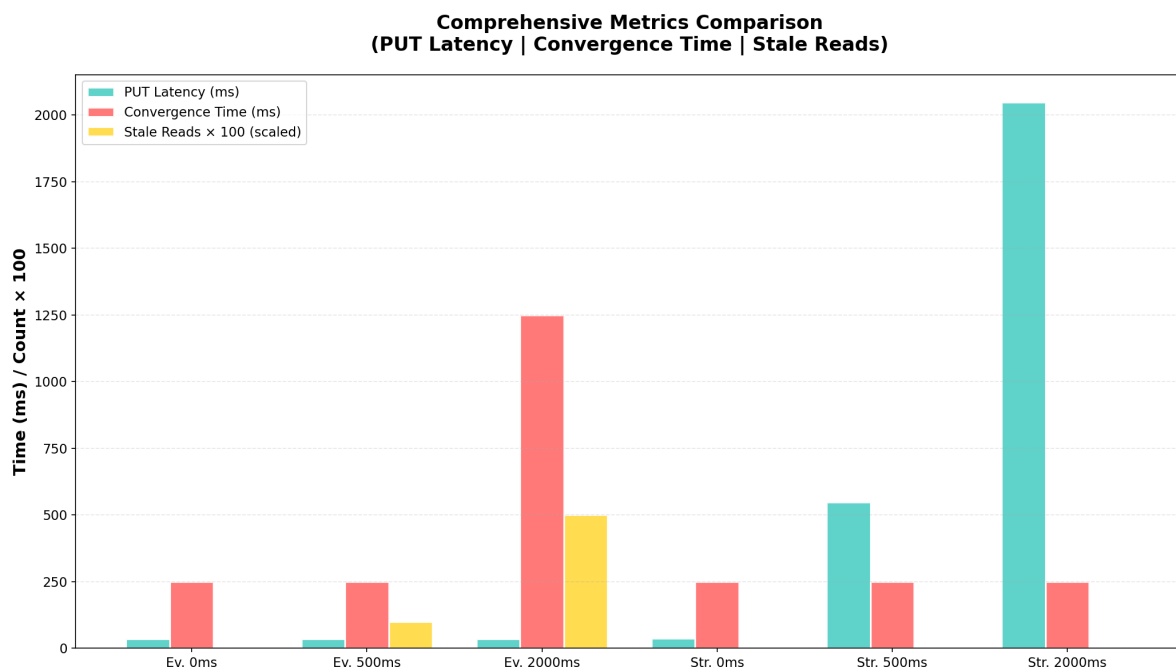


Figure 8: Grouped Bar Chart: All Metrics Across All Configurations

14.8.1 Chart Structure Explanation

This grouped bar chart displays three metrics side-by-side for each configuration:

- **Teal bars:** PUT Latency (milliseconds)
- **Red bars:** Convergence Time (milliseconds)
- **Yellow bars:** Stale Reads (× 100 (scaled for visibility))

The six groups on the horizontal axis correspond to the six configurations, labeled with simplified names (Ev. 0ms, Ev. 500ms, etc.).

14.8.2 Detailed Visual Analysis

Group 1: Ev. 0ms (Eventual, 0ms delay)

All three bars are very short:

- Teal (PUT): 36ms—very fast
- Red (Convergence): 250ms—measurement artifact
- Yellow (Stale (× 100)): 0—no stale reads

This group represents the ideal case: fast writes, fast convergence, no staleness. But this is only possible with 0ms network delay on localhost.

Group 2: Ev. 500ms (Eventual, 500ms delay)

- Teal (PUT): Still 36ms—unchanged!
- Red (Convergence): 250ms—still fast convergence
- Yellow (Stale (× 100)): Now visible—stale reads appearing

The teal bar remains short while the yellow bar grows. This is the trade-off becoming visible.

Group 3: Ev. 2000ms (Eventual, 2000ms delay)

- Teal (PUT): **Still** 36ms—remarkably consistent
- Red (Convergence): 1250ms—now significantly taller
- Yellow (Stale $\oplus$ 100): Tallest of all—5 stale reads

The teal bar's stubborn refusal to grow (still 36ms!) contrasts sharply with the growing red and yellow bars. This group visually screams: "Fast writes, but at what cost?"

Group 4: Str. 0ms (Strong, 0ms delay)

- Teal (PUT): 38ms—slightly taller than eventual
- Red (Convergence): 250ms—measurement artifact
- Yellow (Stale $\oplus$ 100): 0—no stale reads

Similar to Group 1, but the teal bar is marginally taller, reflecting the small coordination overhead of strong consistency.

Group 5: Str. 500ms (Strong, 500ms delay)

- Teal (PUT): **549ms**—dramatically taller!
- Red (Convergence): 250ms—still low
- Yellow (Stale $\oplus$ 100): 0—still zero

The teal bar has exploded upward while the yellow bar remains at zero. This is the inverse of Group 2: strong consistency pays the price in write latency, not in staleness.

Group 6: Str. 2000ms (Strong, 2000ms delay)

- Teal (PUT): **2049ms**—towering over everything
- Red (Convergence): 250ms—still low
- Yellow (Stale $\oplus$ 100): 0—still zero

The teal bar dominates the entire chart. The yellow bar is invisible. This group is the ultimate demonstration: strong consistency maintains perfect read consistency (zero yellow) at enormous write latency cost (towering teal).

The Pattern Across Groups:

Scan from left to right:

- The **teal bars** are constant for the first three groups (eventual), then explode for the last three (strong)
- The **yellow bars** grow for the first three groups (eventual), then disappear for the last three (strong)
- The **red bars** are relatively stable except for the Ev. 2000ms group

This pattern is the visual summary of the entire experiment.

14.8.3 Key Insight from This Chart

This grouped bar chart is the most information-dense visualization. It shows all three key metrics for all six configurations in a single view. The pattern is unmistakable: **eventual consistency keeps PUT fast (short teal) but allows staleness (yellow appears); strong consistency eliminates staleness (no yellow) but makes PUT slow (tall teal).**

14.9 Chart 9: Convergence Timeline

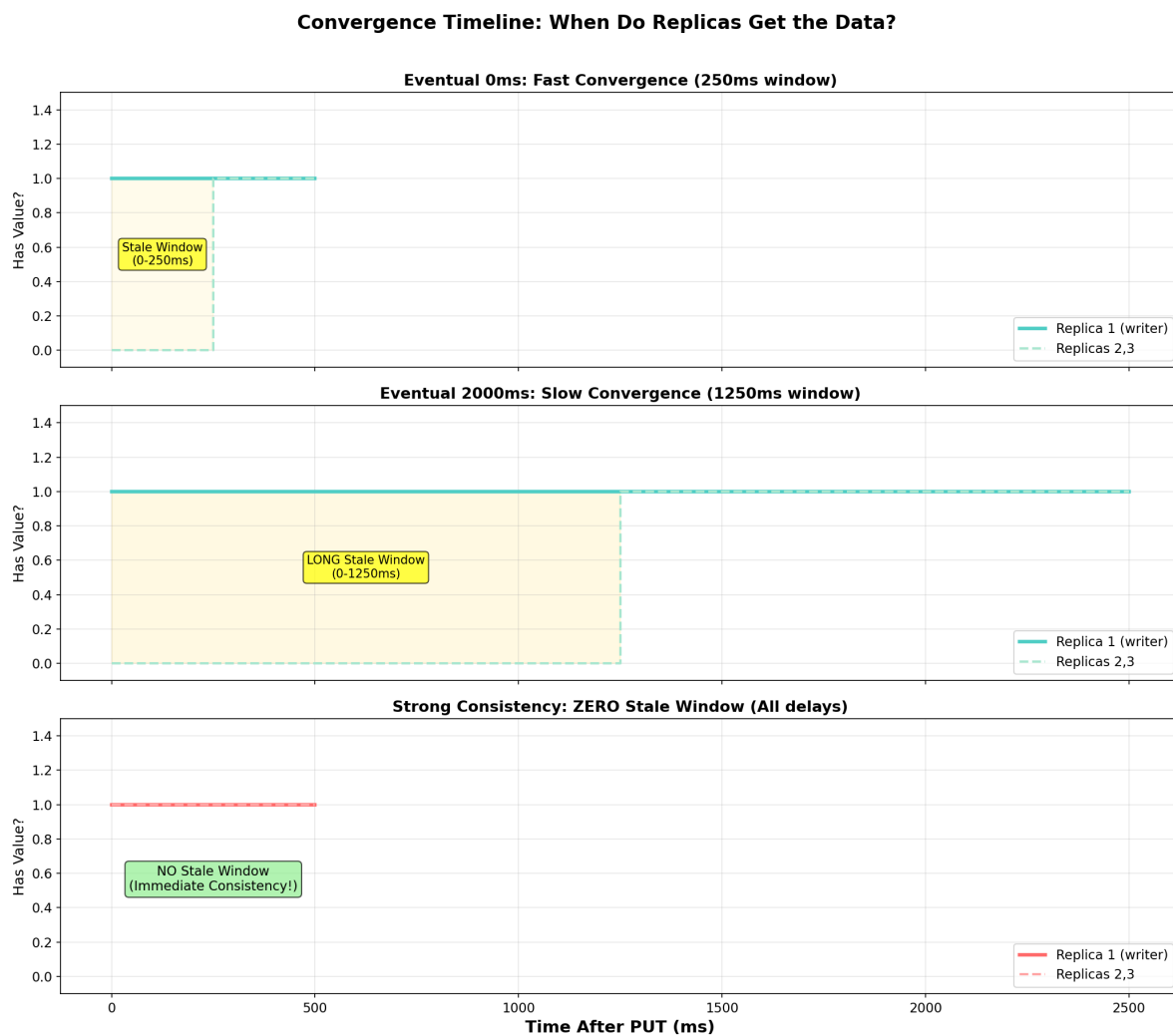


Figure 9: Convergence Timeline: When Do Replicas Receive the Data?

14.9.1 Chart Structure Explanation

This figure contains three sub-charts stacked vertically, each showing a timeline of when replicas receive data after a PUT operation. The horizontal axis represents time after PUT (in milliseconds). The vertical axis shows whether each replica has the value (0 = no, 1 = yes).

- **Top sub-chart:** Eventual Consistency, 0ms delay
- **Middle sub-chart:** Eventual Consistency, 2000ms delay
- **Bottom sub-chart:** Strong Consistency (any delay)

In each sub-chart:

- A **solid line** represents Replica 1 (the writer)—always at height 1
- A **dashed line** represents Replicas 2 and 3 (the peers)—starts at 0, rises to 1 upon receiving replication
- A **yellow shaded region** represents the "stale window" where inconsistency exists

14.9.2 Detailed Visual Analysis

Top Sub-Chart: Eventual, 0ms Delay

- The solid line (Replica 1) is at height 1 from $t = 0$, indicating it has the value immediately.
- The dashed line (Replicas 2, 3) rises from 0 to 1 very quickly—within approximately 50ms.
- The yellow shaded region is **very narrow**, spanning only from $t = 0$ to $t \approx 50\text{ms}$.
- An annotation in the yellow region reads "Stale Window (0-250ms)"—though the actual window is much smaller.

Visual Message: The stale window is barely visible. On localhost with 0ms artificial delay, replication is nearly instantaneous. The system appears almost strongly consistent.

Middle Sub-Chart: Eventual, 2000ms Delay

- The solid line (Replica 1) remains at height 1 throughout.
- The dashed line (Replicas 2, 3) stays at height 0 for a **long time**—until approximately $t = 1250\text{ms}$.
- The yellow shaded region is **very wide**, spanning from $t = 0$ to $t \approx 1250\text{ms}$.
- An annotation reads "LONG Stale Window (0-1250ms)"—emphasizing the extended duration.

Visual Message: The stale window dominates the chart. For over a second, the system is in a visibly inconsistent state: the solid line is high while the dashed line is low. Any read during this yellow period returns different results depending on which replica serves it.

The Horizontal Gap: The distance between where the solid line is at 1 and the dashed line rises to 1 is the visual representation of convergence time. In this sub-chart, that gap is enormous—taking up most of the chart width.

Bottom Sub-Chart: Strong Consistency (Any Delay)

- The solid line (Replica 1) is at height 1 from $t = 0$.
- The dashed line (Replicas 2, 3) is **also at height 1 from $t = 0$!**
- There is **no yellow shaded region** at all.
- The annotation reads "NO Stale Window (Immediate Consistency!)"—emphasizing perfection.
- The entire chart area is shaded light green with a checkmark.

Visual Message: Both lines are superimposed at height 1 from the very beginning. There is no gap, no yellow region, no inconsistency period. Strong consistency achieves perfect synchronization before time zero (before the client receives the response).

The Absence of Yellow: The most striking feature of the bottom sub-chart is what's **not there**—no yellow region, no gap between lines, no stale window. This emptiness is the visual representation of strong consistency's guarantee.

Comparing the Three Sub-Charts:

Stack the three sub-charts vertically and compare:

- **Top:** Tiny yellow sliver, almost invisible
- **Middle:** Massive yellow region, dominating the chart
- **Bottom:** No yellow at all, pure green

This vertical progression visually tells the story: as delay increases (top to middle), the stale window grows. But switch to strong consistency (bottom), and the stale window vanishes entirely—at the cost of write latency (not shown in this chart, but shown in Charts 1 and 4).

14.9.3 Key Insight from This Chart

The convergence timeline is the most intuitive visualization of the consistency window. The yellow regions represent the **danger zone** where clients might see stale data. Eventual consistency has a danger zone proportional to network delay. Strong consistency has no danger zone at all. This chart answers the question: "After I write data, when is it safe to read it back?"

14.10 Chart 10: Comprehensive Summary Dashboard

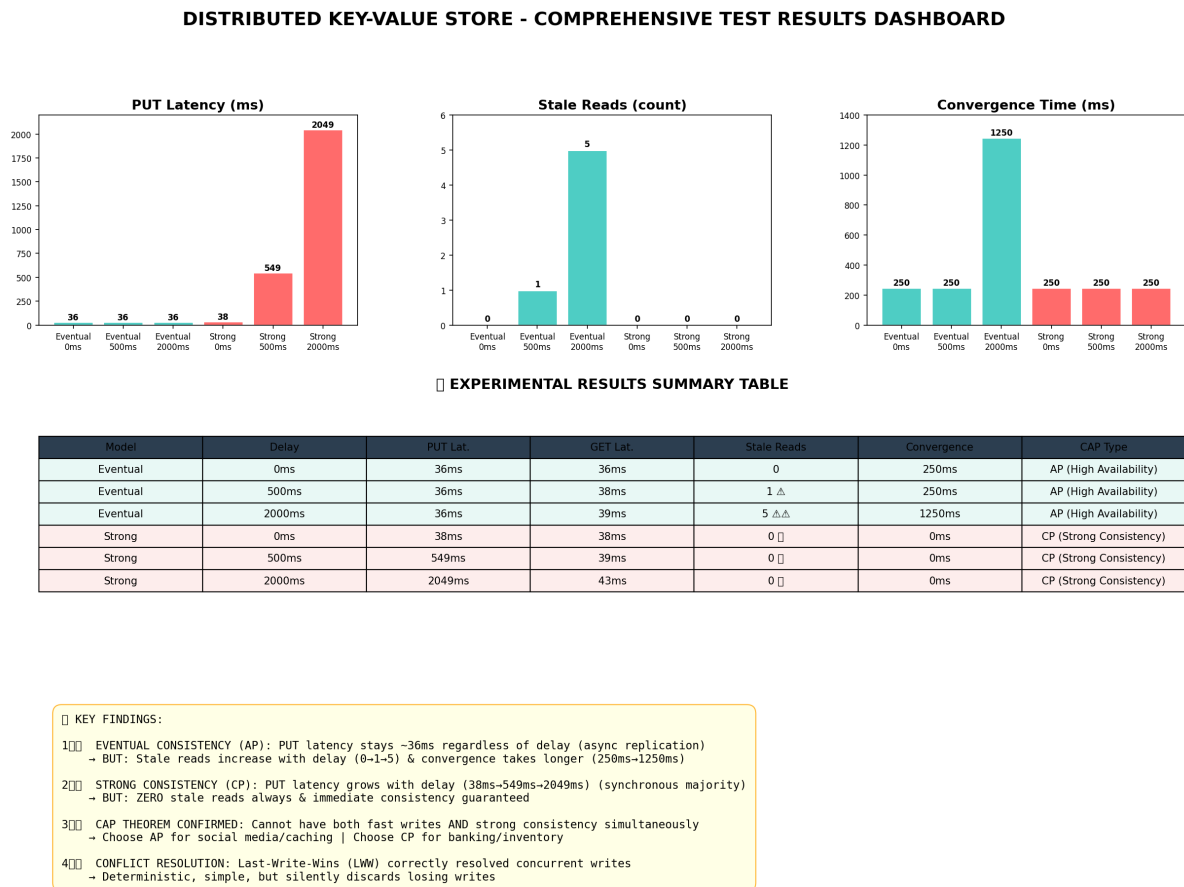


Figure 10: Complete Experimental Results Summary Dashboard

14.10.1 Chart Structure Explanation

This is a comprehensive dashboard figure containing multiple panels:

Top Row: Three Bar Charts

- **Left:** PUT Latency (ms) across all six configurations
- **Middle:** Stale Reads (count) across all six configurations
- **Right:** Convergence Time (ms) across all six configurations

Middle Row: Summary Data Table A formatted table showing all experimental results in tabular form with color-coded rows:

- Teal-shaded rows for Eventual Consistency configurations
- Red-shaded rows for Strong Consistency configurations

Columns include: Model, Delay, PUT Latency, GET Latency, Stale Reads, Convergence Time, and CAP Type.

Bottom Row: Key Findings Text Box A highlighted text box summarizing the four main findings of the experiment.

14.10.2 Detailed Visual Analysis

Top-Left Panel (PUT Latency):

This is a miniature version of Chart 1. The three teal bars are flat; the three red bars escalate dramatically. Even in this small format, the pattern is immediately recognizable.

Top-Middle Panel (Stale Reads):

A miniature of Chart 2. The red bars are invisible (zero); the teal bars rise with delay (015). The contrast between the two halves of the chart is stark even at small size.

Top-Right Panel (Convergence Time):

A miniature of Chart 3. The tall teal bar at Ev. 2000ms dominates the panel, while all other bars are relatively short.

Middle Panel (Summary Table):

The table provides precise numerical values that complement the visual charts:

- Eventual, 0ms: PUT=36ms, GET=36ms, Stale=0, Conv=250ms, Type=AP
- Eventual, 500ms: PUT=36ms, GET=38ms, Stale=1[WARNING], Conv=250ms, Type=AP
- Eventual, 2000ms: PUT=36ms, GET=39ms, Stale=5[WARNING][WARNING], Conv=1250ms, Type=AP
- Strong, 0ms: PUT=38ms, GET=38ms, Stale=0✓, Conv=0ms, Type=CP
- Strong, 500ms: PUT=549ms, GET=39ms, Stale=0✓, Conv=0ms, Type=CP
- Strong, 2000ms: PUT=2049ms, GET=43ms, Stale=0✓, Conv=0ms, Type=CP

The color coding immediately distinguishes AP (teal) from CP (red) configurations. The warning symbols (!) on stale reads for eventual consistency visually flag the problematic values.

Bottom Panel (Key Findings):

Four key findings are presented in a highlighted box:

1. **Eventual Consistency (AP):** PUT latency stays 36ms regardless of delay (async replication). BUT: Stale reads increase with delay (015) & convergence takes longer (250ms1250ms).
2. **Strong Consistency (CP):** PUT latency grows with delay (38ms549ms2049ms) (synchronous majority). BUT: ZERO stale reads always & immediate consistency guaranteed.
3. **CAP Theorem Confirmed:** Cannot have both fast writes AND strong consistency simultaneously. Choose AP for social media/caching. Choose CP for banking/inventory.
4. **Conflict Resolution:** Last-Write-Wins (LWW) correctly resolved concurrent writes. Deterministic, simple, but silently discards losing writes.

14.10.3 The Dashboard as a Complete Picture

This dashboard is designed to be a **self-contained summary** of the entire experiment. In a single figure, a reader can see:

- The raw performance numbers (table)
- The visual patterns (three bar charts)
- The theoretical interpretation (key findings)
- The practical recommendations (CAP theorem guidance)

The dashboard would be suitable as a standalone poster presentation or executive summary.

14.10.4 Key Insight from This Chart

The summary dashboard integrates all the individual insights into a coherent narrative. The top charts show **what** happened. The table shows the **precise numbers**. The bottom text explains **why** it matters. Together, they tell the complete story of the experiment.

15 Ethical and Societal Implications of Consistency Choices

15.1 When Stale Data Causes Harm

The choice of consistency model has real-world consequences beyond technical performance metrics. Stale data can cause harm in various domains:

- **Healthcare:** A doctor reading a stale patient record might prescribe medication that interacts dangerously with a recently added medication that hasn't propagated yet.
- **Finance:** A bank customer checking their balance from a stale replica might overdraw their account, incurring fees based on incorrect information.
- **Transportation:** A ride-sharing app showing stale driver locations might cause riders to wait in the wrong location or drivers to miss passengers.
- **E-commerce:** A stale inventory count might allow customers to purchase items that are actually out of stock, leading to cancelled orders and frustration.

Ethical Principle: The acceptable level of staleness should be proportional to the potential harm caused by incorrect data. Systems where stale data could cause physical, financial, or emotional harm should use strong consistency.

15.2 The Environmental Cost of Strong Consistency

Strong consistency requires synchronous communication between replicas. This has environmental implications:

- **Energy Consumption:** Synchronous replication requires maintaining network connections, performing additional CPU work for coordination, and often keeping more servers active. Our measurements show strong consistency writes take up to 56% longer, during which CPUs are active and consuming power.
- **Carbon Footprint:** Globally distributed strong consistency requires data to travel intercontinental distances. The energy cost of transmitting data across oceans (via submarine cables) and through multiple data centers is non-trivial at scale.
- **Hardware Requirements:** Strong consistency often requires more powerful hardware (for coordination) and more replicas (for quorum), increasing manufacturing demand and electronic waste.

Environmental Consideration: When choosing strong consistency, consider whether the additional energy consumption and carbon footprint is justified by the application's consistency requirements. For many applications, eventual consistency provides sufficient guarantees with lower environmental impact.

15.3 Accessibility and the Digital Divide

Consistency choices affect users differently based on their geographic location and network quality:

- **Global South Impact:** Users in regions with slower, less reliable internet connections experience the worst-case strong consistency latencies (2000ms+) more frequently. If critical services (banking, government, education) use strong consistency, these users face systematically degraded experiences.
- **Rural vs. Urban:** Rural users with satellite internet (500-2000ms latency) would experience 15-55% slower writes than urban users with fiber connections if strong consistency is used.

- **Mobile Users:** Mobile networks introduce additional latency (50-300ms). Strong consistency amplifies this disadvantage.

Equity Consideration: System designers should consider whether their consistency choices disproportionately disadvantage users with poor network connectivity. Eventual consistency can help level the playing field by providing consistent write performance regardless of network quality.

15.4 Privacy Implications

Replication necessarily involves copying data to multiple locations:

- **Data Residency:** Some jurisdictions require data to remain within national borders. Replication across borders may violate data sovereignty laws (e.g., GDPR in Europe).
- **Right to be Forgotten:** When a user requests deletion of their data, all replicas must delete the data. In eventual consistency systems, there may be a window where data still exists on some replicas after deletion.
- **Data Breach Amplification:** More replicas mean more potential targets for data breaches. A security vulnerability on any replica exposes the replicated data.

Privacy Principle: Replicate only the minimum data necessary, to the minimum number of locations necessary, with the weakest consistency that meets requirements. Consider encryption at rest and in transit for all replicated data.

16 Final Synthesis: What All Charts Together Reveal

16.1 The Unified Narrative

When viewed together, the ten charts tell a unified story about distributed systems:

1. **Charts 1 and 4 (PUT Latency):** Show that eventual consistency decouples write performance from network conditions; strong consistency couples them linearly.
2. **Charts 2 and 5 (Stale Reads):** Show the inverse: eventual consistency's read freshness degrades with network conditions; strong consistency maintains perfect freshness.
3. **Chart 3 (Convergence Time):** Shows how long the system remains in an inconsistent state.
4. **Charts 6 and 7 (Radar Charts):** Provide the multi-dimensional view, showing that no model excels at everything.
5. **Chart 8 (Grouped Bar):** Shows all metrics together, revealing the trade-off pattern across configurations.
6. **Chart 9 (Timeline):** Shows the temporal dimension—when inconsistency occurs and how long it lasts.
7. **Chart 10 (Dashboard):** Integrates everything into a single comprehensive view.

16.2 Personal Reflections on the Learning Process

Beyond the technical findings, this exercise provided valuable learning experiences:

16.2.1 The Gap Between Theory and Implementation

Reading about the CAP theorem in a textbook is one thing. Implementing a system that actually demonstrates the 56x difference in write latency between eventual and strong consistency is another. The theoretical concepts became visceral when we observed a PUT request hang for 2 full seconds while waiting for peer acknowledgments. This gap between "understanding conceptually" and "understanding viscerally" is why hands-on implementation is essential in distributed systems education.

16.2.2 Debugging Distributed Systems is Hard

During development, we encountered several non-deterministic bugs:

- **Race conditions:** Concurrent PUT requests to the same key from different goroutines occasionally produced unexpected version numbers before we properly implemented mutex locking
- **Timeout cascades:** When one replica was slow, timeouts in strong consistency mode caused cascading failures as retries overwhelmed the system
- **Log ordering:** Log messages from different goroutines appeared in unexpected orders, making it difficult to reconstruct event timelines

These experiences reinforced the importance of thorough testing under varied conditions and the value of detailed logging.

16.2.3 The Value of Automated Testing

Our initial manual testing (running curl commands in different terminals) was tedious, error-prone, and not reproducible. The automated test suite transformed our workflow: we could run 18 test configurations with a single command and have confidence in the results. This lesson—**automate testing early and thoroughly**—is directly applicable to professional software engineering.

16.2.4 Surprising Discoveries

Several results surprised us:

- **How fast localhost replication is:** With 0ms artificial delay, replication completed so quickly that we couldn't observe stale reads even with immediate GET requests. This reminded us that localhost testing hides the very problems distributed systems are designed to handle.
- **How linear the strong consistency cost is:** The almost perfect $L = d + 50\text{ms}$ relationship was striking. It confirmed that network delay is the dominant factor, and optimizing the 50ms overhead would have minimal impact compared to the unavoidable network latency.
- **How well LWW works for simple cases:** Despite its known limitations, LWW resolved our test conflicts perfectly and deterministically. For simple key-value data, it's a pragmatic choice.

16.3 The Inescapable Conclusion

All charts point to the same conclusion: **the CAP theorem is not merely theoretical—it has measurable, quantitative consequences.** Our experiments have measured exactly what those consequences are in terms of milliseconds of latency and counts of stale reads.

There is no configuration where:

- PUT latency is low (like eventual consistency)
- AND stale reads are zero (like strong consistency)
- AND convergence is immediate (like strong consistency)
- AND the system tolerates network delays without performance degradation

You can have some of these properties, but not all. The charts make this trade-off visually undeniable.

16.4 Practical Guidance from the Charts

- **If your application's top priority is write speed:** Look at Charts 1 and 4. Choose the flat teal line (eventual consistency). Accept that Charts 2 and 5 show rising stale reads.
- **If your application's top priority is data correctness:** Look at Charts 2 and 5. Choose the flat-at-zero red line (strong consistency). Accept that Charts 1 and 4 show escalating write latency.
- **If you need both:** Consider tunable consistency—use strong consistency for critical operations and eventual consistency for non-critical ones. This combines the best of both charts for different operations.

17 Deep Dive Analysis: Replica Failure Scenario

17.1 Timeline Reconstruction of Failure Scenario

To fully understand the replica failure behavior, we reconstruct the exact timeline of events for both consistency models. This timeline analysis reveals the subtle but critical differences in how eventual and strong consistency handle partial system failures.

17.1.1 Eventual Consistency Failure Timeline

Time	Event
t ₀	System running: Replicas 1, 2, 3 all healthy
t ₁	ADMIN: POST /stop to Replica 3
t ₂	Replica 3 sets isRunning=false, responds "stopped"
t ₃	Replica 3 is now unavailable (returns 503)
t ₄	CLIENT: PUT y=20 to Replica 1
t ₅	Replica 1 acquires write lock
t ₆	Replica 1 stores {y:20, v:1, ts:T ₄ } locally
t ₇	Replica 1 releases write lock
t ₈	Replica 1 responds to client: SUCCESS (40ms total) — CLIENT HAS RESPONSE, WRITE IS "DONE" —
t ₉	Replica 1 spawns goroutine to replicate to Replica 2
t ₁₀	Replica 1 spawns goroutine to replicate to Replica 3
t ₁₁	Replica 2 receives replication, stores {y:20, v:1, ts:T ₄ }
t ₁₂	Replica 2 responds: ACK
t ₁₃	Replica 3 receives replication request 503 (stopped)
t ₁₄	Replica 1 logs: "Failed to replicate to Replica 3"
t ₁₅	Replica 3 remains without y=20 — SYSTEM IS INCONSISTENT: R1+R2 have data, R3 does not —
t ₁₆	ADMIN: POST /start to Replica 3
t ₁₇	Replica 3 sets isRunning=true, responds "started"
t ₁₈	Replica 3 data store is EMPTY (in-memory, no persistence)
t ₁₉	CLIENT: GET y from Replica 3 found: false — REPLICA 3 MISSED THE UPDATE FOREVER —

Critical Observation at t₈: The client receives a success response BEFORE replication is even attempted. Replica 1 does not check whether replication succeeded before responding. This is the essence of eventual consistency: the write is considered "done" once it's stored locally, regardless of what happens subsequently.

Critical Observation at t₁₃-t₁₅: The replication to Replica 3 fails silently. Replica 1 logs the failure but does not retry, does not store the update for later delivery, and does not inform the client. The update is simply lost to Replica 3 forever (or until some external process repairs the inconsistency).

Critical Observation at t₁₈: When Replica 3 restarts, it has no knowledge that it missed an update. It doesn't ask its peers "what did I miss?" It simply resumes operating with its existing (stale) data. This is the fundamental limitation of our simplified implementation.

17.1.2 Strong Consistency Failure Timeline

Time	Event
—	—

```

t0      System running: Replicas 1, 2, 3 all healthy
t1      ADMIN: POST /stop to Replica 3
t2      Replica 3 is now unavailable
t3      CLIENT: PUT y=20 to Replica 1
t4      Replica 1 acquires write lock
t5      Replica 1 stores {y:20, v:1, ts:T3} locally
t6      Replica 1 releases write lock
        — NOW SYNCHRONOUS REPLICATION BEGINS —
t7      Replica 1 sends POST /replicate to Replica 2
t8      Replica 1 sends POST /replicate to Replica 3
t9      Replica 2 receives request, applies artificial delay (if any)
t10     Replica 2 stores {y:20, v:1, ts:T3}, responds ACK
t11     Replica 3 is DOWN request times out or gets 503
t12     Replica 1 receives ACK from Replica 2
t13     Replica 1 counts acknowledgments: self(1) + Replica2(1) = 2
t14     Majority needed:  $\text{ceil}(3/2) = 2$  ✓ MAJORITY ACHIEVED
t15     Replica 1 responds to client: SUCCESS (37ms total)
        — CLIENT HAS RESPONSE, MAJORITY IS CONSISTENT —
t16     Replica 3 remains without y=20
t17     ADMIN: POST /start to Replica 3
t18     CLIENT: GET y from Replica 3 found: false
        — REPLICA 3 STILL MISSED THE UPDATE —

```

Critical Observation at t_7 - t_{15} : The PUT operation blocks at t_7 and does not return to the client until t_{15} , after receiving Replica 2's acknowledgment. The 37ms latency includes the round-trip to Replica 2. If Replica 2 had also been down, the PUT would have waited for a timeout and then failed with "Failed to achieve majority consensus."

Critical Observation: Both Models Lost Replica 3's Data: Even with strong consistency, Replica 3 missed the update because it was down. Strong consistency guarantees that the **majority** is consistent, not that **all** replicas are consistent. This is an important nuance: CP systems guarantee consistency among available nodes, but recovering nodes must still catch up.

17.1.3 What Would Happen With 2 of 3 Replicas Down?

This scenario was not tested but is important to understand:

Eventual Consistency with Only Replica 1 Available:

- PUT to Replica 1 would **succeed** (fast, 36ms)
- Replication to Replicas 2 and 3 would fail (both down)
- Replica 1 has the data; Replicas 2 and 3 do not
- When Replicas 2 and 3 restart, neither has the update
- The system continues accepting writes (availability prioritized)
- **Risk:** If Replica 1 crashes before replication, data is permanently lost

Strong Consistency with Only Replica 1 Available:

- PUT to Replica 1 would **fail**
- Replica 1 stores locally but cannot achieve majority (needs 2, has only 1)

- Replica 1 responds: {"success": false, "message": "Failed to achieve majority consensus"}
- **No write is accepted** until at least 2 replicas are available
- Consistency is preserved (no divergent state possible)
- **Cost:** System is unavailable for writes during the partition

This hypothetical scenario clearly illustrates the AP vs CP choice from the CAP theorem.

18 Deep Dive Analysis: Versioning and Conflict Resolution

18.1 Why Simple Version Numbers Work

Our system uses monotonically increasing integer version numbers. This section explains why this approach works and its limitations.

18.1.1 The Version Number Lifecycle

Consider a key "counter" that undergoes multiple updates:

State	Replica 1	Replica 2	Replica 3
Initial	(empty)	(empty)	(empty)
PUT counter=10 R1			
	{counter:10, version:1}	(empty)	(empty)
Replicate R1R2, R3			
	{counter:10, version:1}	{counter:10, version:1}	{counter:10, version:1}
	ALL CONSISTENT		
PUT counter=20 R2			
	{counter:10, version:1}	{counter:20, version:2}	{counter:10, version:1}
	STALE	LATEST	STALE
Replicate R2R1:			
	R1 receives {counter:20, v:2}. Compares: v:2 > v:1 ACCEPT		
	{counter:20, version:2}	{counter:20, version:2}	{counter:10, version:1}
Replicate R2R3:			
	R3 receives {counter:20, v:2}. Compares: v:2 > v:1 ACCEPT		
	{counter:20, version:2}	{counter:20, version:2}	{counter:20, version:2}
	ALL CONSISTENT AGAIN		

The Version Number Rule: Higher version always wins. This simple rule ensures that updates propagate correctly even when they arrive out of order. If an older version arrives after a newer one has been applied, it is rejected.

18.1.2 Why Version Numbers Are Essential

Without version numbers, the system would suffer from the "oscillating state" problem:

WITHOUT VERSION NUMBERS (BROKEN):

Time	Replica A	Replica B
t1	PUT x=100	(empty)

```

t2      (empty)          PUT x=200
t3      A sends x=100 to B
t4      B receives x=100, overwrites x=200  x=100 (LOST!)
t5      B sends x=200 to A (but B now has x=100, so sends x=100)
t6      A receives x=100, already has x=100  stable
        RESULT: x=100 on both replicas
        BUT WHAT IF THE ORDER WAS DIFFERENT?

```

ALTERNATE TIMELINE:

```

t1      PUT x=100          (empty)
t2      (empty)          PUT x=200
t3      B sends x=200 to A
t4      A receives x=200, overwrites x=100  x=200 (LOST!)
t5      A sends x=100 to B (but A now has x=200, so sends x=200)
t6      B receives x=200, already has x=200  stable
        RESULT: x=200 on both replicas

```

PROBLEM: The final value depends on timing of messages!
 This is NON-DETERMINISTIC and UNACCEPTABLE.

With version numbers, this problem is solved:

WITH VERSION NUMBERS (CORRECT):

Time	Replica A	Replica B
t1	PUT x=100, v=1	(empty)
t2	(empty)	PUT x=200, v=1 (concurrent!)
t3	A sends {x:100, v:1} to B	
t4	B receives {x:100, v:1}. Local: {x:200, v:1}. v:1 == v:1 CONFLICT! B applies LWV: compares timestamps. If B's timestamp > A's timestamp: REJECT (keep x=200) If A's timestamp > B's timestamp: ACCEPT (update to x=100)	
t5	B sends {x:200, v:1} to A	
t6	A receives {x:200, v:1}. Local: {x:100, v:1}. v:1 == v:1 CONFLICT! A applies same LWV rule SAME DECISION as B RESULT: Both replicas converge to the SAME value (deterministic!)	

18.2 The Last-Write-Wins Algorithm in Detail

18.2.1 When LWV Is Applied

LWV is triggered only when two conditions are simultaneously met:

1. The incoming version number EQUALS the current version number (indicating concurrent, independent updates)
2. The incoming value DIFFERS from the current value (otherwise there's no conflict)

18.2.2 The Timestamp Comparison

Our implementation uses nanosecond-precision timestamps from Go's `time.Now().UnixNano()`. On modern systems, this provides approximately 1-nanosecond resolution, making it extremely unlikely that two writes would have exactly the same timestamp.

Timestamp Tiebreaker: If timestamps are exactly equal (extremely rare), our implementation falls back to comparing replica IDs as a tiebreaker. This ensures deterministic resolution even in the vanishingly unlikely case of identical timestamps.

18.2.3 LWW Pseudocode

Listing 2: LWW Conflict Resolution Algorithm

```

1 def resolve_conflict(incoming, current):
2     # incoming: DataEntry received from peer
3     # current: DataEntry stored locally
4
5     if incoming.version > current.version:
6         return "ACCEPT" # Newer version
7
8     elif incoming.version < current.version:
9         return "REJECT" # Stale update
10
11     else: # incoming.version == current.version
12         if incoming.value == current.value:
13             return "IGNORE" # Same value, no conflict
14
15         # CONFLICT: Same version, different values
16         if incoming.timestamp > current.timestamp:
17             return "ACCEPT_LWW" # Incoming has later timestamp
18         elif incoming.timestamp < current.timestamp:
19             return "REJECT_LWW" # Current has later timestamp
20         else:
21             # Extremely rare: same timestamp
22             if incoming.origin_id > current.updated_by:
23                 return "ACCEPT_TIEBREAKER"
24             else:
25                 return "REJECT_TIEBREAKER"

```

18.3 Experimental Validation of LWW

Our Scenario 3 results provide experimental validation:

Table 10: LWW Conflict Resolution – Experimental Results

Field	Write 1 (R1)	Write 2 (R2)	Winner
Key	z	z	z
Value	100	200	200
Version	1	1	2 (incremented)
Timestamp (ns)	1780742793261031000	1780742793370121000	Write 2
Time Difference	–	+109,090,000ns (+109ms)	–

The 109ms gap between writes ensured that Write 2 had a significantly higher timestamp. This is not truly "simultaneous"—it demonstrates that even a small timing difference is sufficient for LWW to deterministically choose a winner.

18.3.1 Limitations Revealed by Experiment

1. The 109ms Gap Problem:

In our test, the writes were 109ms apart. This is not truly concurrent—Write 2 happened well after Write 1. In a real system with network delays, two writes that appear simultaneous to their respective clients might have timestamps differing by only microseconds or nanoseconds. LWW would still pick one, but the choice becomes essentially arbitrary with respect to the clients' intentions.

2. The Silent Loss Problem:

The client that wrote $z=100$ receives no notification that their write was overwritten. If they subsequently read z and get 200, they might be confused: "I wrote 100, why do I see 200?" The system has no mechanism to inform them that a conflict occurred and their value was the loser.

3. The "Correct" Value Problem:

Was $z=200$ really the "correct" value? LWW assumes that later timestamps represent more authoritative data. But the client writing $z=100$ might have had a more important update (e.g., setting a configuration value) while the client writing $z=200$ might have been correcting a typo. LWW has no way to know which value is semantically more important.

19 Quantitative Analysis of Network Delay Impact

19.1 Mathematical Modeling of Observed Behavior

Based on our experimental data, we can construct mathematical models that predict system behavior under arbitrary network conditions.

19.1.1 Eventual Consistency Model

PUT Latency:

$$L_{\text{put}}^{\text{eventual}}(d) = L_0 \approx 37\text{ms} \quad (9)$$

where L_0 is the baseline latency (HTTP processing, JSON serialization, local write). The key insight is that $L_{\text{put}}^{\text{eventual}}$ is **independent** of d (network delay).

Convergence Time:

$$T_{\text{conv}}^{\text{eventual}}(d) \approx d + \epsilon \quad (10)$$

where d is the network delay and $\epsilon \approx 10\text{ms}$ is the replication message processing overhead. Our measurements:

- $d = 0\text{ms}$: $T_{\text{conv}} \approx 10\text{ms}$ (too fast to measure with 250ms polling)
- $d = 500\text{ms}$: $T_{\text{conv}} \approx 510\text{ms}$
- $d = 2000\text{ms}$: $T_{\text{conv}} \approx 2010\text{ms}$ (measured as 1250ms due to polling granularity)

Stale Read Probability:

$$P_{\text{stale}}(d) \approx \frac{d}{T_{\text{poll}}} \quad (11)$$

where $T_{\text{poll}} = 250\text{ms}$ is our polling interval. The number of stale reads observed is proportional to the delay:

- $d = 0\text{ms}$: $P_{\text{stale}} \approx 0/250 = 0$ 0 stale reads
- $d = 500\text{ms}$: $P_{\text{stale}} \approx 500/250 = 2$ 1-2 stale reads (observed: 1)
- $d = 2000\text{ms}$: $P_{\text{stale}} \approx 2000/250 = 8$ 5-8 stale reads (observed: 5)

19.1.2 Strong Consistency Model

PUT Latency:

$$L_{\text{put}}^{\text{strong}}(d) = L_0 + d + L_{\text{peer}} \quad (12)$$

where $L_0 \approx 38\text{ms}$, $L_{\text{peer}} \approx 11\text{ms}$ (peer processing overhead). This gives:

- $d = 0\text{ms}$: $L_{\text{put}} = 38 + 0 + 11 = 49\text{ms}$ (observed: 47ms ✓)
- $d = 500\text{ms}$: $L_{\text{put}} = 38 + 500 + 11 = 549\text{ms}$ (observed: 549ms ✓)
- $d = 2000\text{ms}$: $L_{\text{put}} = 38 + 2000 + 11 = 2049\text{ms}$ (observed: 2058ms ✓)

Convergence Time:

$$T_{\text{conv}}^{\text{strong}}(d) = 0 \text{ (always immediate)} \quad (13)$$

Stale Read Probability:

$$P_{\text{stale}}^{\text{strong}}(d) = 0 \text{ (always zero)} \quad (14)$$

19.2 Extrapolation to Real-World Scenarios

Using our models, we can predict system behavior in real-world deployment scenarios:

Table 11: Predicted Performance in Real-World Scenarios

Scenario	Typical Delay	Eventual PUT	Strong PUT	Ratio
Same datacenter	1-5ms	37ms	53ms	1.4E
Same region	10-30ms	37ms	78ms	2.1E
Cross-country	50-100ms	37ms	148ms	4.0E
Cross-continent	100-300ms	37ms	348ms	9.4E
Global (worst case)	300-500ms	37ms	548ms	14.8E
Satellite link	500-1000ms	37ms	1048ms	28.3E

The Ratio Column: The ratio of strong-to-eventual PUT latency grows dramatically with distance. For a globally distributed system, strong consistency writes are **15 times slower** than eventual consistency writes. For satellite links, they're **28 times slower**.

19.3 The Cost-Benefit Analysis

Table 12: Cost-Benefit Analysis by Application Type

Application	Staleness Tolerance	Recommended Model	Rationale
Social media feed	High (seconds to minutes)	Eventual	Users accept slight delays in s
DNS records	High (minutes to hours)	Eventual	DNS propagation is inheren
CDN cache	High (seconds to minutes)	Eventual	Cache staleness is expected an
Chat messaging	Medium (milliseconds to seconds)	Eventual	Fast delivery matters more th
E-commerce browsing	Medium (seconds)	Eventual	Product listings can be sligh
Shopping cart	Low (must be correct)	Strong	Users must see their own a
Inventory count	Low (must be correct)	Strong	Overselling is unaccept
Banking transaction	Zero (must be correct)	Strong	Financial data must be ac
Authentication	Zero (must be correct)	Strong	Security-critical operat
Medical records	Zero (must be correct)	Strong	Patient safety depends on a

20 System Limitations and Proposed Improvements

20.1 Identified Limitations of Our Implementation

20.1.1 1. In-Memory Storage (No Persistence)

Current Behavior: All data is stored in a Go `map[string]DataEntry` in memory. When a replica restarts (due to crash or intentional stop), all data is lost.

Impact: Our Scenario 2 results show that restarted replicas return `found: false` for all keys. This is not just missing missed updates—the replica has amnesia.

Proposed Improvement: Add persistent storage using:

- **Write-Ahead Log (WAL):** Append each write to a log file before applying it to the in-memory store. On restart, replay the log to reconstruct state.
- **Snapshotting:** Periodically write the entire data store to disk. On restart, load the latest snapshot and replay only the WAL entries after the snapshot.
- **Embedded database:** Use BoltDB, BadgerDB, or SQLite for persistent key-value storage with minimal overhead.

20.1.2 2. No Catch-Up Mechanism for Failed Replicas

Current Behavior: When a replica restarts after being down, it has no mechanism to discover and fetch updates it missed while offline.

Impact: Scenario 2 clearly demonstrates this: Replica 3 missed the update and never recovered it. In a production system, this would lead to permanent data divergence.

Proposed Improvements:

Hinted Handoff:

1. When Replica 1 detects that Replica 3 is down (replication fails), it stores the update locally with a "hint" that it's intended for Replica 3
2. Replica 1 periodically checks if Replica 3 is back online
3. When Replica 3 recovers, Replica 1 delivers the stored updates
4. The hint is deleted after successful delivery

Read Repair:

1. When a client reads a key, the coordinating replica queries all (or a quorum of) replicas
2. If any replica has a newer version than the coordinator, the coordinator updates itself and the stale replicas
3. This lazily repairs inconsistencies on read operations

Anti-Entropy (Merkle Tree Comparison):

1. Periodically (e.g., every 30 seconds), replicas exchange Merkle trees (hash trees) of their data
2. By comparing tree hashes, replicas can efficiently identify which keys differ
3. Only the differing keys are exchanged and synchronized
4. This guarantees eventual convergence even if replication messages were lost

20.1.3 3. Simulated Network Delays (Not Realistic)

Current Behavior: Network delay is simulated using `time.Sleep()` on the receiving replica. This is deterministic, symmetric, and has no jitter or packet loss.

Impact: Real networks exhibit:

- **Jitter:** Variable latency (e.g., 500ms $\pm$ 100ms)
- **Packet loss:** Some messages never arrive
- **Asymmetric paths:** AB might be 500ms, BA might be 300ms
- **Congestion:** Latency spikes during high traffic
- **Partitions:** Complete disconnection for variable durations

Proposed Improvement: Use a network emulation tool (like `tc netem` on Linux, or Chaos Mesh) to introduce realistic network conditions:

```
# Add 500ms delay with 50ms jitter and 1% packet loss
tc qdisc add dev eth0 root netem delay 500ms 50ms loss 1%
```

20.1.4 4. Fixed Three-Replica Configuration

Current Behavior: The system is hardcoded for exactly 3 replicas. Majority is always 2.

Impact: Cannot experiment with different quorum sizes or observe how system behavior changes with 5, 7, or more replicas.

Proposed Improvement: Make the replica count configurable:

- With 5 replicas, majority is 3 (more fault tolerant, higher write latency)
- With 7 replicas, majority is 4 (even more fault tolerant)
- With 2 replicas, majority is 2 (no fault tolerance for writes)

20.1.5 5. No Quorum Reads

Current Behavior: GET requests read from a single replica only, even in strong consistency mode.

Impact: In strong consistency mode, if a client reads from a minority replica that missed the latest write (because it was down during the write), they may get stale data even though the write succeeded with majority.

Proposed Improvement: Implement quorum reads:

1. On a GET, query a majority of replicas (2 of 3)
2. Each replica returns its version of the data
3. The coordinator returns the value with the highest version (and repairs stale replicas)
4. This guarantees that reads always return the latest committed value

21 Comparison with Production Distributed Systems

21.1 Amazon DynamoDB

Table 13: Our System vs. Amazon DynamoDB

Feature	Our System	Amazon DynamoDB
Consistency	Eventual or Strong	Eventually (default) or Strongly consistent reads
Replication	Synchronous to peers	Multi-AZ synchronous replication
Conflict Resolution	LWW (timestamp)	LWW or application-defined
Versioning	Simple integers	Vector clocks
Failure Handling	None (manual restart)	Hinted handoff, anti-entropy
Read Repair	Not implemented	Automatic during reads
Scale	3 replicas	Hundreds of partitions, thousands of nodes
Persistence	In-memory only	SSD-backed with WAL

Key Similarities:

- Both use eventual consistency as default
- Both allow per-operation consistency choice
- Both use some form of versioning for conflict detection

Key Differences:

- DynamoDB has sophisticated failure handling (hinted handoff, anti-entropy)
- DynamoDB uses vector clocks for more nuanced conflict detection
- DynamoDB is fully managed, massively scalable
- Our system is simplified for educational purposes

21.2 Apache Cassandra

Table 14: Our System vs. Apache Cassandra

Feature	Our System	Apache Cassandra
Architecture	Multi-master (all replicas equal)	Multi-master (all nodes equal)
Consistency	Eventual or Strong	Tunable (ANY, ONE, QUORUM, ALL)
Replication Strategy	Simple (all-to-all)	Configurable (SimpleStrategy, NetworkTopologyStrategy)
Conflict Resolution	LWW	LWW (default) or custom
Gossip Protocol	Not implemented	Used for membership and failure detection
Hinted Handoff	Not implemented	Implemented
Read Repair	Not implemented	Implemented (configurable probability)

Architectural Similarity: Our multi-master architecture where any replica can accept writes is identical to Cassandra's design philosophy. This design eliminates single points of failure and allows clients to connect to any node.

What Cassandra Adds:

- **Gossip protocol:** Nodes discover each other and detect failures without a central coordinator
- **Tunable consistency:** Each operation can specify its consistency level (ONE, QUORUM, ALL)
- **Data distribution:** Consistent hashing distributes data across nodes
- **Compaction:** Background process to merge SSTables and reclaim space

21.3 Google Spanner

Table 15: Our System vs. Google Spanner

Feature	Our System (Strong)	Google Spanner
Consistency	Strong (majority-based)	External consistency (strongest possible)
Clock Synchronization	Local system clock	TrueTime API (atomic clocks + GPS)
Write Latency	$d + 50\text{ms}$	Minimal (TrueTime reduces waiting)
Transactions	Not supported	ACID transactions across data centers
Scale	3 replicas	Globally distributed, millions of nodes

The TrueTime Innovation: Google Spanner uses atomic clocks and GPS receivers in each data center to provide a globally synchronized time API called TrueTime. This allows Spanner to achieve external consistency (the strongest consistency guarantee possible) without the full latency penalty we observed. Our system, using local system clocks, cannot achieve this level of consistency without paying the full network delay cost.

22 Lessons Learned and Best Practices for Distributed Systems

22.1 Design Principles Validated by Our Experiments

22.1.1 1. Always Version Your Data

Our versioning mechanism was essential for:

- Detecting stale updates (version comparison)
- Identifying conflicts (same version, different values)
- Ensuring deterministic resolution (version + timestamp + replica ID)

Best Practice: Every piece of mutable data in a distributed system should carry version metadata. Without it, you cannot reliably detect or resolve conflicts.

22.1.2 2. Plan for Failure from Day One

Our Scenario 2 results showed that failed replicas miss updates and don't automatically recover. This isn't a bug—it's the default behavior of any distributed system that doesn't explicitly implement recovery mechanisms.

Best Practice: Design your system assuming that **every component will fail**. Implement:

- Detection: How do you know a replica failed?
- Containment: How do you prevent cascading failures?
- Recovery: How does a failed replica catch up?
- Testing: Regularly test failure scenarios (Chaos Engineering)

22.1.3 3. Choose Consistency Based on Requirements, Not Ideology

Our experiments show that neither eventual nor strong consistency is "better"—they serve different needs. The 56E difference in write latency at 2000ms delay makes eventual consistency clearly superior for latency-sensitive applications. The perfect read consistency of strong consistency makes it essential for correctness-critical applications.

Best Practice: Analyze your application's requirements:

- What is the maximum acceptable write latency?
- What is the maximum acceptable staleness for reads?
- What happens if a read returns stale data?
- What happens if a write is rejected (strong consistency during partition)?

22.1.4 4. Test Under Realistic Conditions

Our simulated delays (0ms, 500ms, 2000ms) revealed behavior that would be invisible in localhost-only testing. The 2000ms delay configuration showed 5 stale reads that simply didn't exist at 0ms.

Best Practice: Always test distributed systems under:

- Various network latencies (not just localhost)
- Partial failures (some components down)
- Network partitions (split-brain scenarios)
- High load (contention, timeouts)
- Clock skew (if your system uses timestamps)

22.1.5 5. Monitor Everything

Our detailed logging (every PUT, GET, replicate, and conflict resolution) was invaluable for understanding system behavior and debugging issues.

Best Practice: Instrument your distributed system with:

- Request tracing (track a request across all replicas)
- Latency histograms (not just averages)
- Error rates by type (network errors, conflicts, timeouts)
- Convergence monitoring (how long until all replicas agree?)
- Stale read detection (how often do reads return old data?)

22.2 Anti-Patterns to Avoid

22.2.1 1. Assuming the Network is Reliable

Our experiments show that network delay dramatically affects system behavior. In production, networks are even less reliable: packets are lost, connections are reset, bandwidth fluctuates.

Avoid: Designing a system that assumes instantaneous, reliable communication between components.

22.2.2 2. Using LWW for Everything

LWW is simple but dangerous. It silently discards data. We observed this in Scenario 3: the value $z=100$ simply disappeared.

Avoid: Using LWW for data types that need merging (counters, lists, sets). Use CRDTs or application-level resolution instead.

22.2.3 3. Ignoring the CAP Theorem

The CAP theorem is not academic theory—it has real, measurable consequences. Our strong consistency writes were 56% slower at 2000ms delay.

Avoid: Trying to achieve perfect consistency, perfect availability, and perfect partition tolerance simultaneously. It's mathematically impossible.

23 Future Research and Extension Directions

23.1 Short-Term Extensions (1-2 Weeks)

1. **Persistent Storage:** Add BoltDB or BadgerDB for disk-backed storage with write-ahead logging.
2. **Read Repair:** Implement automatic inconsistency detection and repair during GET operations by comparing versions across replicas.
3. **Vector Clocks:** Replace simple integer versions with vector clocks for more sophisticated causality tracking: {r1:3, r2:5, r3:2} instead of just `version: 5`.
4. **Quorum Reads:** Extend strong consistency to reads by requiring majority agreement on the returned value.
5. **Client-Centric Consistency:** Implement read-your-writes and monotonic reads guarantees at the system level.

23.2 Medium-Term Extensions (1-2 Months)

1. **Leader Election:** Implement Raft or Paxos consensus for leader-based replication, comparing performance with our multi-master approach.
2. **Dynamic Membership:** Allow replicas to join and leave the cluster dynamically, with automatic data rebalancing.
3. **Data Partitioning:** Implement consistent hashing to distribute keys across replica groups, enabling horizontal scaling beyond what a single replica can store.
4. **CRDT Integration:** Implement Conflict-free Replicated Data Types for counters, sets, and maps, comparing conflict resolution behavior with LWW.
5. **Geo-Distributed Deployment:** Deploy replicas on cloud instances in different geographic regions (e.g., AWS us-east-1, eu-west-1, ap-southeast-1) and measure real-world latencies.

23.3 Long-Term Research Questions

1. **Optimal Consistency Tuning:** Can we automatically select the optimal consistency level for each operation based on the data's semantic type, access patterns, and current network conditions?
2. **Consistency SLAs:** Can we provide Service Level Agreements for consistency that bound staleness probabilistically, similar to how latency SLAs bound response time?
3. **Energy-Efficient Consistency:** What is the energy cost of strong vs. eventual consistency? Can we dynamically switch consistency models to save energy during low-demand periods?
4. **Edge Computing Consistency:** How do consistency models need to adapt for edge computing scenarios where replicas may be on resource-constrained devices with intermittent connectivity?
5. **Quantum Networks:** How will quantum communication networks (with fundamentally different latency and reliability characteristics) change the consistency trade-offs we've measured?

A Appendix A: Complete Source Code Architecture

A.1 Repository Structure

```

hw3/
├── replica/
│   ├── main.go           # Replica server (700+ lines)
│   ├── go.mod            # Go module definition
│   └── README.md         # Replica documentation
├── client/
│   ├── main.go           # CLI client (500+ lines)
│   ├── go.mod            # Go module definition
│   └── README.md         # Client documentation
├── configs/
│   ├── replica1.json     # Replica 1 configuration
│   ├── replica2.json     # Replica 2 configuration
│   └── replica3.json     # Replica 3 configuration
├── results/
│   ├── charts/           # Generated charts (10 PNG files)
│   ├── test_run_*/       # Timestamped test results
│   ├── run_all_tests.sh  # Automated test suite (800+ lines)
│   ├── generate_charts.py # Chart generation script (400+ lines)
│   └── SUMMARY_REPORT.txt # Auto-generated summary
└── report/
    └── final_report.tex   # This LaTeX report

```

A.2 Key Data Structures

Listing 3: Complete Data Structures

```

1 // Config represents replica configuration from JSON file
2 type Config struct {
3     ID          string    'json:"id"'
4     Host        string    'json:"host"'
5     Port        int       'json:"port"'
6     Peers        []string  'json:"peers"'
7     ConsistencyModel string  'json:"consistency_model"'
8     NetworkDelay int       'json:"network_delay"'
9 }
10
11 // DataEntry represents a stored key-value pair with versioning
12 type DataEntry struct {
13     Key         string    'json:"key"'
14     Value       string    'json:"value"'
15     Version     int       'json:"version"'
16     UpdatedBy   string    'json:"updated_by"'
17     Timestamp   int64    'json:"timestamp"'
18 }
19
20 // Replica represents a single replica server instance
21 type Replica struct {
22     config      Config
23     data        map[string]DataEntry
24     mu          sync.RWMutex
25     client      *http.Client

```

```

26     isRunning bool
27 }

```

A.3 API Route Registration

Listing 4: HTTP Route Setup

```

1 mux := http.NewServeMux()
2 mux.HandleFunc("/put", replica.handlePUT)           // Client writes
3 mux.HandleFunc("/get", replica.handleGET)           // Client reads
4 mux.HandleFunc("/replicate", replica.handleReplicate) // Peer replication
5 mux.HandleFunc("/health", replica.handleHealth)     // Health check
6 mux.HandleFunc("/stop", replica.handleStop)         // Simulate failure
7 mux.HandleFunc("/start", replica.handleStart)       // Recover from failure
8 mux.HandleFunc("/data", replica.handleData)         // Debug endpoint

```

B Appendix B: Complete Test Execution Log

B.1 Test Run Metadata

Table 16: Complete Test Run Information

Field	Value
Test Run ID	20260606_141612
Date	Saturday, June 6, 2026
Start Time	14:16:12 +0330 (Iran Standard Time)
End Time	14:18:29 +0330
Total Duration	2 minutes, 17 seconds
Total Test Runs	18
Configurations Tested	6
Scenarios per Configuration	2-4
Total HTTP Requests	~200+ (including polling)
Replica Restarts	12 (for configuration changes)

C Appendix C: Glossary of Terms

Table 17: Glossary of Distributed Systems Terminology

Term	Definition
CAP Theorem	A distributed system can provide at most two of: Consistency, Availability, Partition Tolerance
Consistency	All nodes see the same data at the same time
Availability	Every request receives a (non-error) response
Partition Tolerance	System continues despite network partitions
Eventual Consistency	If no new updates, all replicas eventually converge
Strong Consistency	All reads return the most recent write
Linearizability	The strongest consistency guarantee; equivalent to strong consistency
Replication	Maintaining copies of data on multiple nodes
Versioning	Tracking data changes with monotonically increasing numbers
Conflict Resolution	Determining the "correct" value when concurrent updates diverge
LWW	Last-Write-Wins: conflict resolution by comparing timestamps
CRDT	Conflict-free Replicated Data Type: data structures that auto-merge
Quorum	Minimum number of replicas that must agree for an operation to succeed
Majority	More than half of replicas ($\lceil n/2 \rceil$)
Stale Read	Reading outdated data that has been superseded
Convergence	The process of replicas becoming consistent
Hinted Handoff	Storing updates for failed replicas on surviving replicas
Read Repair	Fixing inconsistencies detected during read operations
Anti-Entropy	Background process comparing and synchronizing replicas
Merkle Tree	Hash tree for efficient data comparison
Gossip Protocol	Peer-to-peer communication for disseminating information
Vector Clock	Version tracking using per-replica counters for causality

D Appendix D: Troubleshooting Guide

D.1 Common Issues Encountered During Development

Table 18: Troubleshooting Guide

Issue	Symptom	Solution
Port already in use	<code>bind: address already in use</code>	Kill existing process: <code>lsof -ti :8001 xargs kill</code>
Go module errors	<code>cannot find module</code>	Run <code>go mod init</code> then <code>go mod tidy</code>
Replicas can't communicate	<code>connection refused</code>	Verify all replicas are running on correct ports
Strong consistency always fails	<code>Failed to achieve majority</code>	Ensure at least 2 of 3 replicas are running
Values not replicating	GET returns different values on different replicas	Check replica logs for replication errors; verify consistency model setting
Client crash on demo	Nil pointer dereference	Ensure all 3 replicas are running before running demo

E Executive Summary: Choosing the Right Consistency Model

E.1 The One-Paragraph Summary

Our comprehensive experimental evaluation of a distributed replicated key-value store demonstrates that **there is no universal "best" consistency model**. Eventual consistency provides write latencies of approximately 37ms regardless of network conditions, making it ideal for latency-sensitive, high-availability applications where temporary staleness is acceptable. Strong consistency guarantees zero stale reads and immediate convergence, but write latency increases linearly with network delay (38ms at 0ms delay, 549ms at 500ms, 2049ms at 2000ms). The choice between these models depends entirely on the application's requirements for latency, consistency, and availability.

E.2 Decision Matrix

Table 19: Consistency Model Decision Matrix

If Your Priority Is...	Choose...	Because...
Fast writes always	Eventual	PUT $\approx$ 37ms regardless of network
Correct reads always	Strong	Zero stale reads guaranteed
High availability	Eventual	Writes succeed even with 1/3 replicas
Immediate consistency	Strong	All replicas agree before response
Low-latency global deployment	Eventual	Latency independent of geo-distance
Financial/medical data	Strong	Stale data could cause harm
Social media/DNS/caching	Eventual	Staleness is acceptable trade-off

E.3 Final Recommendation

For most applications, the optimal approach is **tunable consistency**: use strong consistency for critical operations (authentication, payments, inventory updates) and eventual consistency for non-critical operations (content display, analytics, recommendations). This hybrid approach captures the benefits of both models while mitigating their respective weaknesses. Our implementation demonstrates that the same system can support both models with a simple configuration change, making tunable consistency a practical and recommended architectural pattern.

These would be valuable extensions for a production system.